

使用哈希函数构造密码学证明

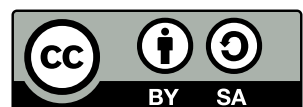
Alessandro Chiesa

Eylon Yogev

译者: Kurt Pan

July 3, 2025

This work is licensed under a Creative Commons “Attribution-ShareAlike 4.0 International” license.



Contents

前言	iv
序言	vi

1 数学背景	1
------------------	---

colback
colback//

I 什么是密码学证明？

2 随机预言机模型	12
3 基本密码学性质	19
4 附加的安全性定义	24
Bibliography	27

前言

简洁证明系统是一种非凡的工具。我们用一个简单的例子来简要解释它们是什么。假设爱丽丝有一个做了一些有趣事情的公共程序 P ，比如会依次输出 π 的十进制数字，首先是 3，然后是 1，然后是 4，依此类推。爱丽丝在她的笔记本电脑上运行这个程序，程序输出了 π 的第一百万位小数，这正好是 5。她非常兴奋，于是告诉了她所有的朋友。爱丽丝的一个朋友鲍勃有点怀疑。他检查了程序 P 的代码，确认它确实是在正确地计算 π 的数字。但他担心爱丽丝只是猜测了第一百万位数字是什么而并没有实际运行程序。鲍勃决定自己验证一下这个计算。他设置好他的笔记本电脑运行 P ，并等待它重新确认爱丽丝的发现。

当鲍勃等待他的笔记本电脑从头开始重新运行整个计算时，他开始觉得这毫无意义。如果他们的朋友卡罗尔也不信任他们中的任何一个，她也将不得不重新运行整个计算以说服自己 P 确实正确运行了。如果大卫不信任他们三个人中的任何一个，他也将不得不重新运行整个计算。这是大量的重复劳动。他们所有人都在一遍又一遍地重做相同的计算。

有没有更好的方法？简洁证明系统提供了一个非凡的解决方案。它使得爱丽丝能够计算出程序 P 正确运行并且其输出如所声称的那样的证明。这个证明是简洁的，意味着它很短且验证速度很快。在实践中，简洁证明只有几千字节长，验证只需几毫秒。这里应当停下来对此感到惊叹：无论爱丽丝计算的是 π 的第一百万位还是第一十亿位数字，证明她计算是正确的证明总是只有几千字节长且只需几毫秒就能验证。

令人惊奇的是，这适用于任意程序 P ，即使它需要爱丽丝提供辅助输入。爱丽丝可以运行程序并发布其输出以及程序正确运行的简洁证明。然后，只需几毫秒的工作，鲍勃、卡罗尔、大卫以及其他任何人都可以检查这个证明，并确信爱丽丝正确地运行了这个程序。他们无需重新运行计算。这个简洁证明甚至可以是零知识的，意思是证明不会透露爱丽丝的辅助输入的任何信息，但它仍然可以让所有人相信程序确实在这个输入上正确运行了。

近年来，简洁证明系统的研究已经发展成为一个庞大的研究领域，许多研究者都贡献了许多美妙的想法。推动这一快速进展的原因之一是这些工具的商业化应用，例如对去中心化系统扩容。另一个原因是这一领域为新的技术创新提供了肥沃的土壤。有多种构造简洁证明的方法，给研究者提供了许多探索的方向。同时，工程师们也有强烈的需求希望构造编程框架，使得非专家也能轻松使用这些工具。总体而言，这是一个由研究者和开发者组成的充满活力的社区，这一领域的工作既令人兴奋又充满乐趣。

本书涵盖了一种特定的简洁证明系统构造方法，称为基于哈希的证明。这些证明系统在概念上是最简单的，不需要复杂的数学知识，因此对广泛的读者群体都具有吸引力。此外，这些证明系统的安全性依赖于哈希函数的相对简单的性质。对于标准的密码学哈希函数，这些性质被认为即使在敌手拥有大规模量子计算机的情况下也能成立。因此，这些证明系统是后量子的：即使在构建出大规模量子计算机之后，它们仍将保持安全。

基于哈希的证明系统的理论已经相当成熟且被透彻理解。然而，直到现在，还没有一本书提供所有的详细信息。这就是本书的目的。本书开始部分精确定义了构成简洁证明系统的元素。接下来，本书转向从几个抽象的信息论对象构建简洁证明系统。一个广泛使用的例子是从一个重要对象——交互式预言机证明 (Interactive Oracle Proof, 简称 IOP) 构造简洁证明系统。该构造完全依赖于哈希函数。书中逐步构造必要的工具，然后清晰描述了该构造及其安全性证明。

本书是对证明系统领域的重大贡献。它适合喜欢对材料进行清晰且精确处理的读者。我相信，一旦你读了这本书，你会想要了解更多关于这个领域的知识。本书是进入简洁证明世界的一个很好的起点，带你踏上一段有趣的旅程。

Dan Boneh
2024 年 4 月
斯坦福大学

序言

“证明对我们的生活至关重要，而对于所有基本的事物，我们应该有所预料，对证明是什么的回答将永远是一个发展中的过程。”

Silvio Micali, *计算可靠证明*

本书讨论的是**密码学证明**，即用于证明和验证计算的正确执行的简洁且零知识的（稍后将详细介绍）协议。密码学证明是计算机科学中的一个基本对象，处于密码学和计算复杂性理论的交汇之处。由于实际应用的推动，在学术界和工业界受到了极大的关注。密码学证明结合了计算理论中一些最优美的思想和最强大的工具，是深奥的数学思想在新技术中起关键作用的一个显著例子。

本书提供了一个基于（理想）哈希函数构造的密码学证明的严格介绍，既是学生自学的教材，也是从业者的参考书。这包括了基于（理想）哈希函数的简洁非交互论证（SNARGs）的著名构造。例如，STARKs（可扩展透明知识论证）就是此类 SNARGs 的一个例子。

最新版本 本书的最新版本可以在以下网站找到：

<https://hash-based-snargs-book.github.io/>

该网站还链接了本书源代码的 Git 库，包括最新的更正和补充内容。我们非常欢迎对本书的评论（无论正面或负面的！），以及任何的更正或建议。您可以直接在 GitHub 上的库里提交 issues 或 PR，或者直接通过电子邮件联系我们。

许可证 本书的源代码（以及本书本身）是根据 Creative Commons Attribution-ShareAlike 4.0 International License (CC BY-SA 4.0) 许可的。简而言之，您可以分享和改编本书的源代码，前提是您给予适当的署名并注明任何更改；此外，从本书衍生的材料必须具有相同的许可证（或与之兼容的许可证）。有关此许可证的更多信息，请参见 <https://creativecommons.org/licenses/by-sa/4.0/>。

动机

重新执行问题 计算机科学的一个基本目标是**计算完整性**：

一方如何验证另一方正确地执行了一个给定的计算？

实现这一目标的一种自然方法是**重新执行计算**，并检查这得到的结果是否与声称的结果相同。

例如，考虑一个计算任务：计算 1 亿以内的质数个数。Alice 通过质数计数算法（例如埃拉托色尼筛法）确定答案是 50,847,534，并发布计算声明“1 亿以内的质数个数是 50,847,534”。Bob 如何检查 Alice 的计算声明是否正确？一个选择是 Bob 重新运行 Alice 使用的算法（或其他质数计数算法），并检查 1 亿以内的质数是否确实是 50,847,534。

然而，在许多情况下，重新执行**不是一个选项**。

- 一方可能无法承担（或没有足够的资源）重新执行与计算声明相关的计算。例如，使用最先进的质数计数算法在 2022 年计算 10^{29} 以内的质数个数要花费 90 多核年和超过 1TB 的峰值内存使用；¹ 这样的计算不易重新执行。
- 此外，计算可能涉及无法与他人共享的私有输入，因此无法重新执行计算。以下是一个涉及 RSA 密钥对的例子。回忆一下，RSA 密钥对包括一个包含两个质数（从合适的分布中抽样）的私钥和一个包含它们乘积的公钥；这样的密钥对用于加密方案和基于 RSA 假设的数字签名方案。考虑声明“我知道两个质数，它们的乘积是 RSA 公钥 N ”。揭示这两个质数可以让其他人轻松验证该声明。然而，这样做会泄露与公钥对应的私匙。

于是我们面临重新执行问题：

如何在不重新执行的情况下确保计算完整性？

乍一看这似乎是不可能的。计算复杂性中的分层定理指出，一般来说，没有办法减少确定性计算输出所使用的时间或空间。² 在不重新执行的情况下确保计算完整性似乎是一个难以实现的目标。

避免重新执行 令人惊讶的是，通过要求提出计算声明的一方同时提供该声明的计算完整性的密码学证明，就可以避免重新执行。（特别地，这规避了前面提到的限制，以及在此尚未提及的其他限制。）非正式地，一方可以通过**证明者算法**生成密码学证明来认证计算的正确性。随后，任何人都可以通过**验证算法**检查密码学证明，而非重新执行原始计算。安全性适用于资源适当受限的敌手（稍后会详细介绍）。

但是，密码学证明的验证如何“优于”重新执行呢？这是由于密码学证明可以实现的优良性质。

- **简洁性。**验证算法可以比重重新执行原始计算快得多。这种特性，称为简洁性，使得计算能力弱的一方可以验证计算复杂的计算的正确性。
- **零知识。**密码学证明不会泄露任何计算中使用的私有输入的信息。这种特性称为零知识，使得其他人可以验证涉及秘密的计算的正确性。（这些秘密只有执行计算并生成密码学证明的一方知道。）

满足（某种形式的）简洁性的密码学证明通常称为**简洁证明**，如果是非交互式的，则称为 **SNARG**。一个流行的概念是 **zkSNARK**，即**零知识简洁非交互知识论证**。这些概念将在本书中引入并定义。

应用 密码学证明通常用于学术研究，设计密码学原语、协议和系统。更令人兴奋的是，密码学证明已经在学术界之外找到了应用：它们在许多无法重新执行的场景中得到了广泛的实际应用。截至写作时，它们是应用密码学中最热门的话题之一。

例如，密码学证明在设计安全分布式系统（如区块链）中起着关键作用。它们被用于实现可扩展性和隐私的目标，简要介绍如下。

- **可扩展性。**简洁性性质通过将昂贵的计算移出网络节点，从而增加对等网络中的交易吞吐量。智能合约系统（例如，以太坊）中的计算既慢又昂贵：每个计算步骤都由网络中的每个节点重新执行，因此网络受限于最慢的节点，用户为每个计算步骤支付高昂的价格。一类称为“基于证明的 rollups”的架构将计算移出链外，用户将其计算请求发送给聚合者，聚合者定期生成关于用户计算批次的密码学证明；然后智能合约系统验证密码学证明并授权一个

¹ 参见 <https://oeis.org/A006880> 和 <https://www.mersenneforum.org/showthread.php?t=20473>

² 例如，确定性多带图灵机分层定理指出，对于每个时间可构造函数 $t(n)$ ，复杂性类 $\text{DTIME}(t(n))$ 严格大于复杂性类 $\text{DTIME}(o(\frac{t(n)}{\log t(n)}))$ 。换句话说，有些计算任务的输出可以在时间 $t(n)$ 内确定，但不能在时间 $o(\frac{t(n)}{\log t(n)})$ 内确定。其他资源和计算模型也存在其他分层定理。

反映批次中所有计算的状态转换。简洁性确保检查密码学证明比检查它所证明的计算便宜得多。因此，让网络中的每个节点仅检查密码学证明要便宜得多，并且使系统能够扩展到更多的用户和更大的计算。

- **隐私。**零知识性质允许设计具有强用户隐私的对等网络。非正式地，零知识性质使得网络中的一方能够广播关于涉及其私有输入的计算的密码学证明。例如，计算可能授权从一方向另一方转移数字资产，但不透露这些方的身份或数字资产的类型和数量。这种隐私保护功能对密码货币和其他去中心化金融应用至关重要，因为交易通常涉及高度敏感的信息，不应向整个网络公开。

这些及其他应用在学术界、工业界和政府的多个社区中引起了对密码学证明的极大兴趣。这种更广泛的兴趣需要关于密码学证明的材料更易于访问和系统化，以有效地满足超出少数专家的受众的需求。

范围

密码学证明通常由两个构建模块组成：一种信息论上的概率证明，以及一种将概率证明映射为密码学证明的密码学变换。对这两种模块的多种选择，导致了性质各异的密码学证明体系，这一广阔领域并非本书焦点。本书追求深度：详尽论述一类基于理想哈希函数的密码学证明，省略其他证明的讨论。

具体而言，本书对**基于理想哈希函数的密码学证明**进行系统、严格的阐述。涵盖基本构造，给出诸多安全性质的完整证明，并讨论实用优化。安全归约附带明确误差界，便于实践中参数设定。大多数分析在常数因子内达到紧界，改进了文献中零散不全之处。全书统一术语与符号，以便突出各构造之间的关联。

“纯”随机预言机设置 我们考虑一个简洁的信息论设置：假设哈希函数为真正随机的函数（不同输入映射到相互独立的随机输出），并且所有方共用同一个**随机预言机**。³该设置称为**随机预言机模型**（ROM），在密码学中被广泛使用。

更具体地，我们聚焦“纯”ROM：构造仅依赖随机预言机的密码学证明（无其他密码假设）。此设置近似信息论安全：对手可计算能力无限，仅受查询次数有界的限制。

本书所有构造均为从某种概率证明到 ROM 中的（交互式或非交互式）密码学证明的变换。概率证明本身的设计与分析（常依赖计算复杂性与抽象代数）超出本书范围；我们将其视作构造密码学证明的起点。

为何如此聚焦？ 选择本书内容有以下优势：

- **简单性**。当哈希函数建模为随机预言机后，构造与分析仅需基本概率论与离散数学，无需抽象代数等繁重背景；故 ROM 下构造是已知最简单的密码学证明之一。
- **稳定性**。密码学证明领域更新迅速、整体尚未成熟；而基于 ROM 的构造已被深入研究，未来变化不会偏离现有范式。
- **安全性**。多数密码学依赖结构性假设（如格、群），存在被未来算法攻破的风险；ROM 仅依赖无结构的随机函数，且实用中多用无结构的哈希函数近似，长期安全性更稳健，并具备良好的后量子抗性。
- **透明设置**。公共参数仅为哈希函数选择，无需可信第三方或多方仪式，即可实现透明设置，这使得 STARK 等技术在实践中易于部署。

目标 现有文献对基于 ROM 的密码学证明多零散不全，本书旨在提供系统、可审计的资源，详述安全定义、归约过程和优化细节，为实战系统的安全审计与参数化部署提供依据，并增强社区对该技术的信心。

补充资源 若欲了解其他密码学证明，可参阅：

- Alessandro Chiesa 的概率证明课程和练习（FOPP Summer School [CG21; Chi23]）。
- ZKProof 社区参考文档 [ZKP22]，综述现代零知识证明构造与定义。
- Justin Thaler 的专著 [Tha22] 及在线课程 <https://zk-learning.org/>，概览多种密码学证明与实践考量。

³这与仅假设哈希函数满足具体属性（如抗碰撞性）不同：仅有抗碰撞性不足以实现简洁非交互式论证。

全书结构

所需背景 大多数简洁论证构造需多领域背景，但仅针对 ROM 的构造只需本科水平的离散概率与算法知识。本书第 1 章涵盖所有基础事实，熟悉该章内容的读者可无需其他资料通读全书。

章节安排 第 1 章介绍符号与数学基础。其余章节分为七大部分：

- *Part I*: 什么是密码学证明？：介绍 ROM 和论证定义，形式化自适应安全性。
- ??: ??: Σ -协议到 NARG 的 Fiat–Shamir 变换。✚
- ??: ??: 交互式证明到 NARG 的多轮 Fiat–Shamir 变换。✚
- ??: ??: 基本承诺方案与 Merkle 承诺方案。✚
- ??: ??: PCP 到简洁交互式与非交互式论证（Kilian 与 Micali 变换）。✚
- ??: ??: IOP 到简洁论证（BCS 变换）。✚
- ??: ??: 参数选择、优化和预处理支持（COS 变换）等实践话题。✚

依赖关系见图 1，变换汇总见图 2。

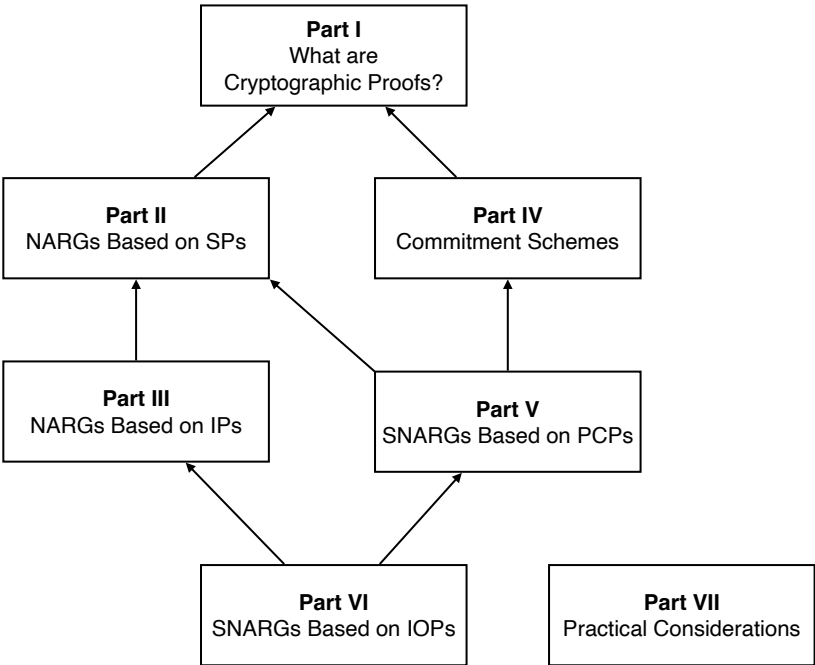


Figure 1: 本书不同部分之间的依赖关系。（NARG 表示非交互式论证，SNARG 表示简洁非交互式论证。）

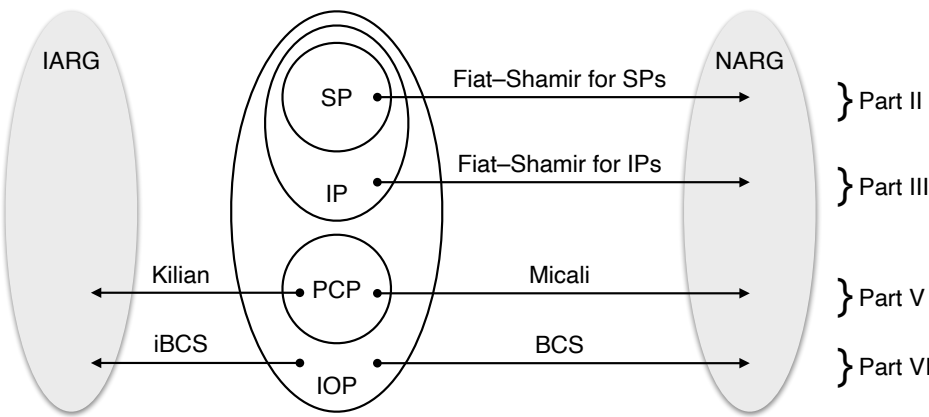


Figure 2: 本书中所有密码学证明的构造都是应用于某种概率证明（SP、IP、PCP 或 IOP）的变换。此图列出了我们研究的变换，显示了哪些构造交互式论证（IARG），哪些构造非交互式论证（NARG）。

如何使用本书

目标读者 本书适用于：

- 学生：本科生与研究生可借此掌握 ROM 下简结论证的核心原理与安全归约技巧。
- 实践者：系统工程师可参考构造细节与具体安全界，进行系统审计与参数部署。

教育者亦可在课程中使用本书，例如：

- 密码学导论课程：第 I 的安全定义，以及 Fiat-Shamir、Kilian 变换等基础内容。
- 密码学证明研究生课程：本书的 ROM 变换构造可作为清晰参考。

致谢

本书得益于以下机构资助：

- 以太坊基金会 (Ethereum Foundation)
- Forte Labs
- Protocol Labs
- Provable

亦感谢 StarkWare Industries 在部分阶段的支持。

特别感谢多年来在学术与产业界给予我们指导的各位同仁，以及以下对本书早期稿件提出宝贵意见的评审（按字母排序）：Gal Arnon、Shany Ben-David、Zijing Di、Giacomo Fenzi、Ziyi Guan、Shai Keidar、Mathias Marty、Amit Sharabi、William Wang、Yuxi Zheng。

感谢版本库中提交 pull request 的各位，及始终支持我们的家人。

1 数学背景

我们介绍本书中使用的基本数学记号和事实。

1.1 符号约定

集合 我们用 $|S|$ 表示集合 S 的基数（即元素个数）；如果 S 的元素个数为无限，则 $|S| := \infty$ 。我们用 $\mathbb{R}$ 表示实数集，用 $\mathbb{N}$ 表示正整数集 $1, 2, \dots$ ，并用 $[n]$ 表示正整数集 $1, 2, \dots, n$ 。对于 $k \in 0, 1, \dots, n$ ，我们用 $\binom{[n]}{k}$ 表示所有从 $[n]$ 中选取 k 个元素的子集所组成的集合；这类子集的数量为 $\binom{n}{k} := \frac{n!}{k!(n-k)!}$ 。我们用 $x_1, \dots, x_n$ 表示包含元素 $x_1, \dots, x_n$ 的集合；该集合的基数为 $x_1, \dots, x_n$ 中不同元素的个数（重复元素只计一次）。给定一个集合 S ，我们记 $x_{i \in S}$ 表示这样一个集合：对每个 $i \in S$ ，都包含元素 x_i ；例如， $x_{i \in [n]} = x_1, \dots, x_n$ 。

元组 元组是元素的有序列表。我们用 $(x_i)_{i \in [n]}$ 表示元组 $(x_1, \dots, x_n)$ ；更一般地， $(x_i)_{i \in S}$ 表示一个元组，其条目根据给定集合 S 的惯例顺序进行索引。元组中重复的元素会在相应位置多次出现。通过嵌套括号可以得到元组的元组；例如， $((x_i, y_i, z_i))_{i \in [n]}$ 表示元组的元组 $((x_1, y_1, z_1), \dots, (x_n, y_n, z_n))$ 。如果 x 是一个条目由集合 S 索引的元组，则对每个 $i \in S$ ， $x[i]$ 表示 x 的第 i 个条目；更一般地，对于子集 $T \subseteq S$ ， $x[T]$ 表示将 x 中索引在 T 内的条目取出后构成的元组。例如，如果 $x = (x_i)_{i \in 1,4,7}$ ，则 $x[7] = x_7$ ， $x[1, 7] = (x_i)_{i \in 1,7}$ 。

字符串 设 Σ 为一个有限字母表（即有限集合）。在 Σ 上的字符串是一个定义在某个有序集合 D 上的函数 $x: D \rightarrow \Sigma$ ，其取值属于 Σ ；特别地， x 是一个元组 $(x_i)_{i \in D}$ ，其中每个条目 x_i 都是字母表 Σ 的元素。通常 $D = [n]$ ($n \in \mathbb{N}$)，此时我们可以通过列举符号的序列来书写字符串；例如，001 表示字符串 $x: [3] \rightarrow \{0, 1\}$ ，其中 $x_1 = 0$ 、 $x_2 = 0$ 、 $x_3 = 1$ 。我们用 $\parallel$ 表示字符串连接；例如 $00\parallel 1$ 表示字符串 001。

对于每个 $n \in \mathbb{N}$ ，我们用 Σ^n 表示所有长度为 n 的 Σ 上的字符串的集合：

$$\Sigma^n := \{x: [n] \rightarrow \Sigma\} = \{(x_i)_{i \in [n]} : \forall i \in [n], x_i \in \Sigma\}.$$

我们用 $\Sigma^* := \cup_{n \in \mathbb{N}} \Sigma^n$ 表示所有有限长度 Σ 上的字符串的集合。

给定一个有序集合 D ，我们用 Σ^D 表示所有长度为 $|D|$ 且条目由 D 的元素索引的 Σ 上的字符串的集合：

$$\Sigma^D := \{x: D \rightarrow \Sigma\} = \{(x_i)_{i \in D} : \forall i \in D, x_i \in \Sigma\}.$$

例如，如果 $\Sigma = \{0, 1\}$ 且 $D = 1, 8, 9$ ，则字符串 $x \in \Sigma^D = \{0, 1\}^{1,8,9}$ 可以具有以下取值： $x[1] = 0$ 、 $x[8] = 1$ 、 $x[9] = 0$ 。

位长 二进制字符串是字母表 $\Sigma = \{0, 1\}$ 上的字符串。我们用 $\text{len}(x)$ 表示二进制字符串 $x \in \{0, 1\}^*$ 的长度；例如 $\text{len}(001) = 3$ 。如果 x 不是二进制字符串，则 $\text{len}(x)$ 表示将 x 适当表示为二进制字符串后的长度。例如，若 $x \in \Sigma^n$ ，则可通过将 x 中的每个符号表示为一个 $\lceil \log_2 |\Sigma| \rceil$ 位的二进制字符串，得到 $\text{len}(x) = n \cdot \lceil \log_2 |\Sigma| \rceil$ 。若 $x \in \Sigma^D$ ，则将 x 视为由 $n = |D|$ 对 (i, x_i) 组成的列表，其中每对由一个 $\lceil \log_2 |D| \rceil$ 位的二进制字符串（编码索引 i ）和一个 $\lceil \log_2 |\Sigma| \rceil$ 位的二进制字符串（编码符号 x_i ）组成，从而 $\text{len}(x) = n \cdot (\lceil \log_2 |D| \rceil + \lceil \log_2 |\Sigma| \rceil)$ 。

抽样 我们用 $a \leftarrow D$ 表示按照分布 D 抽样得到 a 的过程。类似地，给定一个有限集合 S ，我们用 $a \leftarrow S$ 表示按照对 S 上的均匀分布抽样得到 a 的过程。

渐近符号 我们使用标准的渐近符号。例如，给定两个函数 $f, g: \mathbb{N} \rightarrow \mathbb{R}$ ，我们分别用 $f(n) = O(g(n))$ 和 $f(n) = \Omega(g(n))$ 表示存在常数 $c > 0$ ，使得对所有足够大的 n ，都有 $f(n) \leq c \cdot g(n)$ 和 $f(n) \geq c \cdot g(n)$ 。此外，我们用 $f(n) = o(g(n))$ 和 $f(n) = \omega(g(n))$ 分别表示对每个常数 $c > 0$ ，当 n 足够大时，均有 $f(n) \leq c \cdot g(n)$ 和 $f(n) \geq c \cdot g(n)$ 。我们还使用符号 $\text{poly}(n)$ 表示某个未指明的多项式上界函数，即 $n^{O(1)}$ ；更一般地， $\text{poly}(n_1, n_2, \dots)$ 表示 $(n_1 + n_2 + \dots)^{O(1)}$ 。

语言与关系 关系 $\mathcal{R}$ 是由实例-证据对 (x, w) 所组成的集合。对应的语言 $\mathcal{L}(\mathcal{R})$ 是那些存在证据 w 使得 $(x, w) \in \mathcal{R}$ 的实例 x 的集合。我们用 $|x|$ 表示实例的大小；这可以是描述 x 所需的比特数（即 $\text{len}(x)$ ）或其他适用于关系 $\mathcal{R}$ 的度量。

算法 我们非正式地将算法称为在适当计算模型上运行的过程。如果我们不关心算法的时间或空间复杂度（例如当考虑无界对手时），则底层计算模型无关紧要。（无论是单带图灵机、多带图灵机、寄存器机或其他均可。）如果我们讨论算法的时间或空间复杂度，则假设算法在类似现代计算机的寄存器机上运行（具有有限指令集并能对具有适当字长的内存进行读写随机访问）。我们同时考虑确定性算法和（访问一定长度随机位串的）随机化算法。

如果一个算法的运行时间由其输入大小的某个多项式上界，则称该算法在多项式时间内运行。（若算法有多个输入，则考虑它们大小之和。）

交互式算法 如果算法 A 在多次输入-输出调用之间保持状态，则称 A 为交互式。对于输入序列 $a_1, a_2, a_3, \dots$ ， A 的输出如下：

$$b_1 \leftarrow A(a_1), ; b_2 \leftarrow A(a_2), ; b_3 \leftarrow A(a_3), ; \dots$$

有时，将一次执行传递到下一次执行的状态显式化会更有帮助，此时我们将算法 A 视为无状态：

$$(b_1, \text{aux}_1) \leftarrow A(a_1), ; (b_2, \text{aux}_2) \leftarrow A(\text{aux}_1, a_2), ; (b_3, \text{aux}_3) \leftarrow A(\text{aux}_2, a_3), ; \dots$$

黑盒 有时我们描述一种算法，它将其他算法作为输入，并且仅依赖于它们的输入-输出功能（通过在特定输入上运行并获得相应输出），而不使用它们的具体实现。我们称此时这些算法被用作黑盒。

设 A 和 B 为算法。若 A 在某些输入上多次调用 B ，并且除此之外不“查看” B 的实现细节，则称 A 将 B 用作黑盒，记为 $A(\mathcal{O}_B)$ 。更正式地说，若对于任意表示相同功能的两个算法 B_1 和 B_2 ，都有 $A(\mathcal{O}_{B_1}) = A(\mathcal{O}_{B_2})$ ，则满足上述定义。

若 B 是交互式算法，则定义 $\mathcal{O}_B$ 为 B 的无状态表示的黑盒，即给定 B 的内部状态和输入，返回相应输出及新的内部状态的函数的黑盒。具体地， $A(\mathcal{O}_B)$ 可以在输入序列 $a_1, a_2, a_3, \dots$ 上与 B 交互，如下所示：

$$(b_1, \text{aux}_1) \leftarrow \mathcal{O}_B(a_1), ; (b_2, \text{aux}_2) \leftarrow \mathcal{O}_B(\text{aux}_1, a_2), ; (b_3, \text{aux}_3) \leftarrow \mathcal{O}_B(\text{aux}_2, a_3), ; \dots$$

在上述黑盒调用序列中存在一个微妙的技术细节。从 $(B$ 的无状态表示中) 传递给 A 的内部状态 $(\text{aux}_i)_i$ 可能包含关于算法 B 的非黑盒信息（例如 B 的代码）；此外，这些内部状态可能很大（例如，如果 B 效率低下，则可能呈指数规模），这会影响 A 的效率。然而， A 只对 B 的输入-输出行为感兴趣，我们希望确

一般情况称为容斥原理，它提供了一种将 n 个事件 $E_1, \dots, E_n$ 的并集的概率表示为它们所有交集概率（带符号）之和的方法。

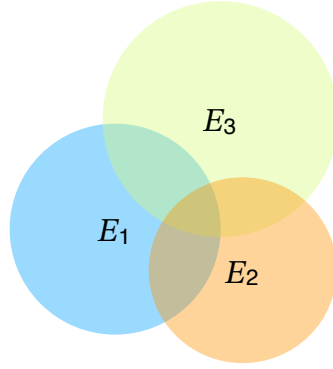


Figure 1.1: 事件 E_1, E_2, E_3 的两两和三重交集示意图。

引理 1.1.1 (inclusion – exclusion principle). 对于任意 n 个事件 $E_1, \dots, E_n$, 有

$$\Pr[\vee_{i \in [n]} E_i] = \sum_{k \in [n]} \left((-1)^{k+1} \cdot \sum_{\substack{I \subseteq [n] \\ \text{with } |I|=k}} \Pr[\wedge_{i \in I} E_i] \right).$$

上述引理中的等式可能不便于使用, 因为它涉及了大量项。下面关于 $\vee_{i \in [n]} E_i$ 的概率的陈述更易使用: 它分别在事件 E_i 的个体概率和它们两两交集 $E_i \wedge E_j$ 的概率基础上给出了一个下界和上界。

引理 1.1.2 (simple inclusion – exclusion principle). 对于任意 n 个事件 $E_1, \dots, E_n$,

$$\sum_{i \in [n]} \Pr[E_i] - \sum_{\substack{i, j \in [n] \\ \text{with } i \neq j}} \Pr[E_i \wedge E_j] \leq \Pr[\vee_{i \in [n]} E_i] \leq \sum_{i \in [n]} \Pr[E_i].$$

Proof. 我们对事件数量 n 进行归纳证明。

基例: $n \in \{1, 2, 3\}$ 。当 $n = 1$ 时, 由于引理中的各表达式均为 $\Pr[E_1]$, 结论显然成立。至于 $n = 2$ 和 $n = 3$ 的情况, 我们在引理之前的讨论中已分别加以证明。

归纳步骤: $n > 1$ 。假设对于任意 $n' \in [n - 1]$, 该引理都已成立。定义事件

$$E^* := \vee_{i \in [n-1]} E_i.$$

由对 $n' = n - 1$ 的归纳假设, 可得

$$\sum_{i \in [n-1]} \Pr[E_i] - \sum_{\substack{i, j \in [n-1] \\ \text{with } i \neq j}} \Pr[E_i \wedge E_j] \leq \Pr[E^*] \leq \sum_{i \in [n-1]} \Pr[E_i]. \quad (1.1)$$

首先证明上界。对事件 E^* 和 E_n 应用 $n' = 2$ 的归纳假设, 并结合 Equation 1.1, 得到

$$\Pr[E^* \vee E_n] \leq \Pr[E^*] + \Pr[E_n] \leq \sum_{i \in [n-1]} \Pr[E_i] + \Pr[E_n] = \sum_{i \in [n]} \Pr[E_i].$$

接下来证明下界。将已证上界应用于事件集合 $\{E_i \wedge E_n\}_{i \in [n]}$, 得到

$$\Pr\left[\bigvee_{i \in [n]} (E_i \wedge E_n)\right] \leq \sum_{i \in [n]} \Pr[E_i \wedge E_n]. \quad (1.2)$$

再对 E^* 和 E_n 应用 $n' = 2$ 的归纳假设, 结合 Equations 1.1 and 1.2, 可得

$$\begin{aligned} & \Pr[E^* \vee E_n] \\ & \geq \Pr[E^*] + \Pr[E_n] - \Pr[E^* \wedge E_n] \\ & \geq \left(\sum_{i \in [n-1]} \Pr[E_i] - \sum_{\substack{i, j \in [n-1] \\ \text{with } i \neq j}} \Pr[E_i \wedge E_j] \right) + \Pr[E_n] - \Pr[E^* \wedge E_n] \\ & = \sum_{i \in [n]} \Pr[E_i] - \sum_{\substack{i, j \in [n-1] \\ \text{with } i \neq j}} \Pr[E_i \wedge E_j] - \Pr\left[\bigvee_{i \in [n]} (E_i \wedge E_n)\right] \\ & \geq \sum_{i \in [n]} \Pr[E_i] - \sum_{\substack{i, j \in [n] \\ \text{with } i \neq j}} \Pr[E_i \wedge E_j]. \end{aligned}$$

□

我们通过涉及随机变量的表达式来描述事件。在本书中, 随机变量取值为二进制字符串或实数。在后者情况下, 二元随机变量指仅取 0 或 1 值的随机变量。

Random variables 如果 X 是一个取值于有限集合 $S \subseteq \mathbb{R}$ 的随机变量, 则其期望定义为:

$$\mathbb{E}[X] := \sum_{x \in S} x \cdot \Pr[X = x].$$

随机变量 X 和 Y 独立当且仅当对任意 x, y 有

$$\Pr[X = x \wedge Y = y] = \Pr[X = x] \cdot \Pr[Y = y].$$

更一般地, 随机变量 $X_1, \dots, X_n$ 独立当且仅当对任意 $x_1, \dots, x_n$ 有

$$\Pr\left[\bigwedge_{i \in [n]} X_i = x_i\right] = \prod_{i \in [n]} \Pr[X_i = x_i].$$

大数定律指出, 给定一系列相互独立且同分布的随机变量 $\{X_i\}_{i \in \mathbb{N}}$, $\mathbb{E}[X]$ 是 $\frac{1}{n} \sum_{i \in [n]} X_i$ 在 $n \rightarrow \infty$ 时的极限。 $\frac{1}{n} \sum_{i \in [n]} X_i$ 与 $\mathbb{E}[X]$ 的偏离随 n 变化的情况由浓缩不等式刻画。其中一个例子是以下(加性) Chernoff 界。

引理 1.1.3 (Chernoff bound). 令独立的二元随机变量 $X_1, \dots, X_n$ 满足 $\Pr[X_i = 1] = p_i$ 。令 $X := \sum_{i \in [n]} X_i$ 和 $\mu := \mathbb{E}[X] = \sum_{i \in [n]} p_i$ 。对于任意 $\delta \in (0, 1)$, 有

$$\Pr[|X - \mu| \geq \delta \cdot \mu] \leq 2 \cdot e^{-\frac{\mu \delta^2}{3}}.$$

Statistical distance 我们以全变差距离来度量两个随机变量的统计距离，如下定义。

定义 1.1.4. 取值于有限集合 S 的随机变量 X 和 Y 之间的统计距离定义为

$$\Delta(X, Y) := \frac{1}{2} \sum_{a \in S} |\Pr[X = a] - \Pr[Y = a]|. \quad (1.3)$$

等价地，统计距离也可定义为

$$\Delta(X, Y) := \max_{S' \subseteq S} |\Pr[X \in S'] - \Pr[Y \in S']|. \quad (1.4)$$

我们将证明上述两种定义 (Equations 1.3 and 1.4) 是等价的。

论断 1.1.5. 使用 Equation 1.3 给出的定义与使用 Equation 1.4 给出的定义等价。

Proof. 定义子集

$$S_0 := \{a \in S : \Pr[X = a] \geq \Pr[Y = a]\},$$

并令其补集为 $S_1 := S \setminus S_0$ 。注意

$$\Pr[X \in S_0] + \Pr[X \in S_1] = 1, \quad \Pr[Y \in S_0] + \Pr[Y \in S_1] = 1.$$

因此

$$\Pr[X \in S_0] - \Pr[Y \in S_0] = \Pr[Y \in S_1] - \Pr[X \in S_1]. \quad (1.5)$$

可以证明 S_0 或 S_1 最大化 $\max_{S' \subseteq S} |\Pr[X \in S'] - \Pr[Y \in S']|$ 。然后，利用 Equation 1.5，得到

$$\Pr[Y \in S_1] - \Pr[X \in S_1] = \max_{S' \subseteq S} |\Pr[X \in S'] - \Pr[Y \in S']| = \Pr[X \in S_0] - \Pr[Y \in S_0].$$

最后，我们有

$$\begin{aligned} \Delta(X, Y) &:= \frac{1}{2} \sum_{a \in S} |\Pr[X = a] - \Pr[Y = a]| \\ &= \frac{1}{2} \left(\sum_{a \in S_0} (\Pr[X = a] - \Pr[Y = a]) + \sum_{a \in S_1} (\Pr[Y = a] - \Pr[X = a]) \right) \\ &= \frac{1}{2} (\Pr[X \in S_0] - \Pr[Y \in S_0] + \Pr[Y \in S_1] - \Pr[X \in S_1]) \\ &= \Pr[X \in S_0] - \Pr[Y \in S_0] \\ &= \max_{S' \subseteq S} |\Pr[X \in S'] - \Pr[Y \in S']|. \end{aligned}$$

□

论断 1.1.6. 统计距离函数具有以下性质：

1. 对任意随机变量 X , $\Delta(X, X) = 0$ 。
2. 对任意随机变量 X, Y , $0 \leq \Delta(X, Y) \leq 1$ 。
3. 对任意随机变量 X, Y, Z ,

$$\Delta(X, Z) \leq \Delta(X, Y) + \Delta(Y, Z).$$

Proof. 前两条性质由定义即得。我们证明第三条如下：

$$\begin{aligned}
 \Delta(X, Z) &= \frac{1}{2} \sum_a |\Pr[X = a] - \Pr[Z = a]| \\
 &= \frac{1}{2} \sum_a |(\Pr[X = a] - \Pr[Y = a]) + (\Pr[Y = a] - \Pr[Z = a])| \\
 &\leq \frac{1}{2} \sum_a |\Pr[X = a] - \Pr[Y = a]| + \frac{1}{2} \sum_a |\Pr[Y = a] - \Pr[Z = a]| \\
 &= \Delta(X, Y) + \Delta(Y, Z).
 \end{aligned}$$

□

论断 1.1.7. 对于任意随机变量 X, Y 和定义在取值域 S 上的函数 f ，有

$$\Delta(f(X), f(Y)) \leq \Delta(X, Y).$$

Proof.

$$\begin{aligned}
 \Delta(f(X), f(Y)) &= \frac{1}{2} \sum_{b \in f(S)} |\Pr[f(X) = b] - \Pr[f(Y) = b]| \\
 &= \frac{1}{2} \sum_{b \in f(S)} \left| \sum_{a \in f^{-1}(b)} (\Pr[X = a] - \Pr[Y = a]) \right| \\
 &\leq \frac{1}{2} \sum_{b \in f(S)} \sum_{a \in f^{-1}(b)} |\Pr[X = a] - \Pr[Y = a]| \\
 &\leq \frac{1}{2} \sum_{a \in S} |\Pr[X = a] - \Pr[Y = a]| \\
 &\leq \Delta(X, Y).
 \end{aligned}$$

□

假设我们有一个随机变量 X 和两个依赖于 X 的随机变量家族 $\{Y(x)\}_x$ 和 $\{Y'(x)\}_x$ 。定义随机变量 Y 为先从 X 取样 x ，再从 $Y(x)$ 中取样； Y' 类似。若

$$\Pr_{x \leftarrow X} [\Delta(Y(x), Y'(x)) \geq \delta] \leq \epsilon,$$

则

$$\Delta((X, Y), (X, Y')) \leq \delta + \epsilon.$$

论断 1.1.8. 如上所述，若 $\Pr_{x \leftarrow X} [\Delta(Y(x), Y'(x)) \geq \delta] \leq \epsilon$ ，则

$$\Delta((X, Y), (X, Y')) \leq \delta + \epsilon.$$

Proof. 定义

$$S := \{x : \Delta(Y(x), Y'(x)) \geq \delta\}.$$

则

$$\begin{aligned}
\Delta((X, Y), (X, Y')) &= \frac{1}{2} \sum_{x, y} |\Pr[(X, Y) = (x, y)] - \Pr[(X, Y') = (x, y)]| \\
&= \frac{1}{2} \sum_{x, y} |\Pr[X = x](\Pr[Y(x) = y] - \Pr[Y'(x) = y])| \\
&= \sum_x \Pr[X = x] \cdot \Delta(Y(x), Y'(x)) \\
&= \sum_{x \in S} \Pr[X = x] \cdot \Delta(Y(x), Y'(x)) + \sum_{x \notin S} \Pr[X = x] \cdot \Delta(Y(x), Y'(x)) \\
&\leq \sum_{x \in S} \Pr[X = x] \cdot 1 + \sum_{x \notin S} \Pr[X = x] \cdot \delta \\
&\leq \epsilon + \delta.
\end{aligned}$$

□

论断 1.1.9. 令 X 和 Y 为同一概率空间上的随机变量, E_1 和 E_2 为两事件且 $\Pr[E_1] = \Pr[E_2]$ 。则

$$\Delta(X, Y) \leq \Delta((X | E_1), (Y | E_2)) + \Pr[\overline{E_1}].$$

Proof. 设 S 为这两个随机变量的取值域。对任意子集 $S' \subseteq S$, 由全概率公式有:

$$\begin{aligned}
&|\Pr[X \in S'] - \Pr[Y \in S']| \\
&= \left| \Pr[X \in S' | E_1] \cdot \Pr[E_1] + \Pr[X \in S' | \overline{E_1}] \cdot \Pr[\overline{E_1}] \right. \\
&\quad \left. - \Pr[Y \in S' | E_2] \cdot \Pr[E_2] - \Pr[Y \in S' | \overline{E_2}] \cdot \Pr[\overline{E_2}] \right| \\
&= \left| \Pr[E_1](\Pr[X \in S' | E_1] - \Pr[Y \in S' | E_2]) \right. \\
&\quad \left. + \Pr[\overline{E_1}](\Pr[X \in S' | \overline{E_1}] - \Pr[Y \in S' | \overline{E_2}]) \right| \\
&\leq \Pr[E_1] |\Pr[X \in S' | E_1] - \Pr[Y \in S' | E_2]| \\
&\quad + \Pr[\overline{E_1}] |\Pr[X \in S' | \overline{E_1}] - \Pr[Y \in S' | \overline{E_2}]| \\
&\leq |\Pr[X \in S' | E_1] - \Pr[Y \in S' | E_2]| + \Pr[\overline{E_1}].
\end{aligned}$$

因此, 由等价定义 Equation 1.4, 得

$$\begin{aligned}
\Delta(X, Y) &:= \max_{S' \subseteq S} |\Pr[X \in S'] - \Pr[Y \in S']| \\
&\leq \max_{S' \subseteq S} |\Pr[X \in S' | E_1] - \Pr[Y \in S' | E_2]| + \Pr[\overline{E_1}] \\
&= \Delta((X | E_1), (Y | E_2)) + \Pr[\overline{E_1}].
\end{aligned}$$

□

1.2 概率实验

我们引入用于定义这些实验上随机变量和事件的记号。

- 我们记 $\{x \mid x \leftarrow A\}$ 表示根据随机算法 A 采样得到字符串 x 的随机变量。
- 我们记 $\Pr[f(x) = 1 \mid x \leftarrow A]$ 表示当 x 根据随机算法 A 采样时, 对于给定的布尔函数 f , 事件 $f(x) = 1$ 的概率。
- 我们记 $\Pr[D(x) = 1 \mid x \leftarrow A]$ 表示当 x 根据随机算法 A 采样, 并且 D 是随机算法时, 事件 $D(x) = 1$ 的概率。在此情况下, 概率在 A 和 D 的随机性上取。

有时我们在表达式内部隐式地定义算法 A 或布尔函数, 使用伪代码和逻辑表达式。例如, 我们可以写

$$\{(y, z) \mid \begin{array}{l} y \leftarrow B \\ z \leftarrow C(y) \end{array}\}$$

表示随机变量 $x := (y, z)$, 由算法 A 实现: 先根据 B 采样得到 y , 再以 y 为输入根据 C 采样得到 z , 并输出对 (y, z) 。我们也可以写

$$\Pr[y = z \mid \begin{array}{l} y \leftarrow B \\ z \leftarrow C(y) \end{array}]$$

表示如上采样得到的随机变量 $x = (y, z)$ 满足布尔函数 $f(x) = 1$ (当且仅当 $y = z$) 的概率。

本书中概率实验的主要目的是通过指定对手的行为并给出其输出的“获胜条件”来描述密码方案的安全性属性。

1.3 辅助等式 (不等式)

我们陈述本书中将反复使用的若干等式和不等式。

引理 1.3.1 (二项式系数的界). 对于任意 $n, k \in \mathbb{N}$ 且 $1 \leq k \leq n$,

$$\left(\frac{n}{k}\right)^k \leq \binom{n}{k} \leq \left(\frac{e \cdot n}{k}\right)^k.$$

引理 1.3.2 (等差级数). 对于任意 $a, d \in \mathbb{R}$,

$$\sum_{i=1}^n (a + (i-1)d) = na + \frac{n(n-1)}{2}d.$$

引理 1.3.3 (等比级数). 对于任意 $r \in (0, 1)$ 和 $a \in \mathbb{R}$,

$$\sum_{i=1}^n a r^{i-1} = a \frac{r^n - 1}{r - 1}.$$

引理 1.3.4. 令 $\delta_1, \dots, \delta_n \in [0, 1]$ 且满足 $\sum_{i=1}^n \delta_i = 1$ 。则

$$\sum_{i=2}^n \delta_{i-1} \cdot \delta_i \leq 1/4.$$

Proof. 我们首先注意到要给出 1 的上界相对容易。由于每个 $\delta_i \leq 1$, 可得

$$\sum_{i=2}^n \delta_{i-1} \cdot \delta_i \leq \sum_{i=2}^n \delta_i \leq 1.$$

下面的证明致力于给出更紧的 $1/4$ 上界。

第一步, 我们证明当除三个变量外其余均为零时, 表达式取得最大值。令索引 $i \in [n]$ 是使 $\delta_{i-1} + \delta_{i+1}$ 最大的最小下标。对于任意 $j \in [n] \setminus \{i-1, i, i+1\}$ 且 $\delta_j > 0$, 我们将 δ_j 的全部权重移动到 δ_i 并展示该操作不会减小表达式。定义新的序列

$$\delta'_k := \begin{cases} 0, & \text{若 } k = j; \\ \delta_i + \delta_j, & \text{若 } k = i; \\ \delta_k, & \text{其他情况.} \end{cases}$$

然后, 原式与新序列下的式子之差仅在含 i 或 j 的项上。我们需验证

$$\delta_{j-1}\delta_j + \delta_j\delta_{j+1} + \delta_{i-1}\delta_i + \delta_i\delta_{i+1} \leq \delta'_{j-1}\delta'_j + \delta'_j\delta'_{j+1} + \delta'_{i-1}\delta'_i + \delta'_i\delta'_{i+1}.$$

注意 $\delta'_j = 0$ 且 $\delta'_i = \delta_i + \delta_j$, 故不等式化简为

$$\delta_{j-1} + \delta_{j+1} \leq \delta_{i-1} + \delta_{i+1},$$

这正是 i 的选择所保证的。因此, 除 $\{i-1, i, i+1\}$ 外, 其余 $\delta_k = 0$ 。

在此基础上, 原式化简为

$$\delta_{i-1}\delta_i + \delta_i\delta_{i+1} = \delta_i(\delta_{i-1} + \delta_{i+1}) = \delta_i(1 - \delta_i) \leq \frac{1}{4},$$

完成证明。 □

Part I

什么是密码学证明？

Overview

我们介绍随机预言机模型 (random oracle model) 并讨论随机预言机的基本密码学性质。然后我们介绍本书研究的主要对象：随机预言机模型中的论证 (arguments)。我们通过证明关于随机预言机模型中论证的若干基本观察来建立直观认识。最后我们描述多个随机预言机的设置，这可以通过单个随机预言机来实现，我们常用它来简化讨论。

2	随机预言机模型	12
2.1	定义	12
2.2	通过惰性采样进行模拟	14
2.3	通过惰性采样进行模拟	15
2.4	伪随机函数 vs. 随机预言机	15
2.5	优点与缺点	17
2.6	多个随机预言机的设置	17
3	基本密码学性质	19
3.1	不可预测性	19
3.2	抗预像性	20
3.3	抗碰撞性	21
3.4	交互式论证	22
4	附加的安全性定义	24
4.1	知识可靠性	24
4.2	零知识	25
	Bibliography	27

2 随机预言机模型

Hash 函数是一种基础的密码学原语，它可以将任意输入映射到看起来随机的固定大小输出。一个流行的研究基于 Hash 函数的密码协议的工具就是 *random oracle model* (简称 ROM)。在该模型中，Hash 函数被假设为理想的，即它将每个输入都映射到独立于其它输入输出的随机结果。这一假设通过为所有参与方（诚实方和恶意方）提供对一个称为随机预言机的随机函数的查询访问来建模。

本书中所研究的所有密码学证明构造都在 ROM 中。在本章中，我们将介绍 ROM 的基本符号和定义，并讨论在 ROM 中进行密码学研究的优缺点。

2.1 定义

随机预言机 对于 $\sigma \in \mathbb{N}$ ，我们用 $\mathcal{U}(\sigma)$ 表示所有形如

$$f: \{0,1\}^* \rightarrow \{0,1\}^\sigma$$

的函数上的均匀分布。等价地，如果 f 从 $\mathcal{U}(\sigma)$ 中采样，那么对于每个输入 x ，都有

$$y := f(x)$$

是一个均匀随机的 σ 位字符串（对于每个输入独立采样）。我们称从 $\mathcal{U}(\sigma)$ 中采样得到的 f 为输出大小为 σ 的随机预言机。

遵循 Section 1.1 中的采样符号，我们将从 $\mathcal{U}(\sigma)$ 中采样 f 表示为

$$f \leftarrow \mathcal{U}(\sigma).$$

随机函数 f 是对真实世界中 Hash 函数的理想化，因为它将 $\{0,1\}^*$ 中的每个输入映射到独立于其它输入输出的随机结果；相比之下，任何通过多项式时间算法（从某个分布中采样）可计算的 Hash 函数都无法满足如此强的性质。¹

预言机算法 从 $\mathcal{U}(\sigma)$ 中采样得到的随机预言机 f 无法作为算法的输入，因为 f 的大小是无限的。相反，算法被赋予对 f 的查询访问权。具体来说，预言机算法是一种在接收常规输入之外，还能对预言机进行多次查询的算法。该操作包括指定一个给预言机的输入（一个查询），并在随后的计算步骤中接收相应的预言机输出（一个回答）。预言机算法可以是确定性的，也可以是随机的（它可以使用与预言机随机回答无关的私有随机性）。

对于算法 A 、预言机 f 和输入 a ，我们用

$$b \leftarrow A^f(a)$$

来表示： A 在拥有对 f 的查询访问权且接收输入 a 时输出 b ；如果 A 是随机的（即拥有自己的私有随机性），则 b 可能是一个随机变量。

对于 $t \in \mathbb{N}$ ，如果一个预言机算法最多进行 t 次查询（不论回答如何），则称其为 t -查询算法。

注意，如果算法 A 的运行时间为 T ，那么 A 只能在大小至多为 T 的输入 x 上对预言机 f 进行查询。具体而言，对 A 的执行而言， f 中只有大小为 $2^T \cdot \sigma$ 的有限部分是相关的。

¹Hash 函数的描述熵至多受限于多项式，而像随机预言机那样回答查询则需要任意大的熵。

模型 ROM 可以通过以下定义形式化描述。

定义 2.1.1. 输出大小为 $\sigma \in \mathbb{N}$ 的 ROM 是这样一种模型：所有参与方（诚实方与恶意方）都是预言机算法，并且它们都获得对同一个函数

$$f: \{0, 1\}^* \rightarrow \{0, 1\}^\sigma$$

的查询访问，该函数从分布 $\mathcal{U}(\sigma)$ 中采样得到。

ROM 的关键点在于，每个参与方都访问同一个随机函数。这不同于每个参与方都访问独立采样随机函数的模型；实际上，那种替代模型是平凡的，因为每个参与方都可以通过惰性采样（如 Section 2.3 所述）高效地模拟自己的随机函数。换句话说，ROM 确保所有参与方对每个可能的查询都能获得相同的随机回答。

在 ROM 中对密码协议进行安全分析时，会根据所考虑的对手类别而有所不同。

- **有查询次数上界的对手。**在此设置下，我们约束对手对随机预言机的查询次数，但不限制其其他资源（时间、空间、随机性等）。我们用整数符号 $t \in \mathbb{N}$ 表示对手的查询上界。直观地，随着 t 增大， t -查询对手破坏协议安全的能力也增强。
- **有时间上界的对手。**在此设置下，我们约束对手的运行时间，并且特别约束其对预言机的查询次数（因为一次查询消耗一个时间单位）。

（更一般地，也可以考虑对其他资源（如内存消耗）进行约束，这将导致相应的设置。）

有查询次数上界的对手的设置可以被视为“纯粹”的 ROM，其中协议只能且必须利用同一份随机函数。而有时间上界的对手的设置则允许额外利用计算难题假设（例如单向函数的存在）。特别地，在有查询次数上界的对手设置下获得的任何结果，都直接适用于有时间上界的对手设置（但反之则不然）。

本书中所有 arguments 的构造都在有查询次数上界的对手设置下（“纯粹”ROM），因此在讨论 ROM 时，默认即指该设置。

其他概念 以下概念有助于描述和分析 ROM 下的协议。

- **查询编码。**随机预言机 f 可以接受任意查询 $x \in \{0, 1\}^*$ 。有时我们将查询 x 定义为由其他二进制字符串组成的元组。例如， $x = (x_1, x_2)$ ，其中 x_1, x_2 是具有特定长度的二进制字符串；在这种情况下，我们隐式使用一个无歧义的编码将元组 (x_1, x_2) 表示为 $\{0, 1\}^*$ 中的一个二进制字符串。有时我们在元组中包含整数（例如 $x = (3, x')$ ），并假定该整数通过某个规定长度的二进制字符串进行表示（通常由协议指定）。
- **查询-回答轨迹。**有时我们分析算法 A 与预言机 f 之间的查询-回答轨迹。我们使用符号

$$b \stackrel{\text{tr}}{\leftarrow} A^f(a)$$

来表示：tr 是算法 A 在输入 a 上与 f 之间进行的有序查询-回答对的列表；如果 A 是随机的，则 tr 是一个随机变量。我们用 $|\text{tr}|$ 表示轨迹 tr 中的查询-回答对数量。

- **查询长度。**算法 A 可以在不同长度的输入上对预言机 f 进行查询。一个查询 x 的查询长度定义为 $\text{len}(x)$ （即二进制字符串 x 的位数）。例如，如果算法对 001, 0101, 010 进行查询，则该算法进行了两个长度为 3 的查询和一个长度为 4 的查询。

Remark 2.1.2 (输入长度固定的预言机). 有时随机预言机会以更受限的方式定义, 即其域固定输入长度而非允许任意长度输入。设 $\mathcal{U}(\lambda \rightarrow \sigma)$ 为所有形如

$$f: \{0, 1\}^\lambda \rightarrow \{0, 1\}^\sigma$$

的函数上的均匀分布, 那么输入长度为 λ 、输出长度为 σ 的随机预言机就是从 $\mathcal{U}(\lambda \rightarrow \sigma)$ 中采样得到的函数。从 $\mathcal{U}(\lambda \rightarrow \sigma)$ 中采样的函数大小为 $2^\lambda \cdot \sigma$, 而从 $\mathcal{U}(\sigma)$ 中采样的函数大小则为无限。除本书中少数例外, 我们通常考虑从 $\mathcal{U}(\sigma)$ 中采样的随机预言机 (即可接受任意长度输入的预言机)。这样可以简化构造和分析。

Remark 2.1.3 (有限样本空间). 我们将随机预言机描述为一个从 $\mathcal{U}(\sigma)$ 中采样的函数

$$f: \{0, 1\}^* \rightarrow \{0, 1\}^\sigma$$

, 这将使其成为一个从无限样本空间中采样的无限大小对象。我们这样做仅为记号上的方便, 因为在本书中考虑的所有构造 (及其相关的安全实验) 中, 所有相关的预言机查询都有明确的最大长度上界。因此, 足以考虑从有限样本空间采样的有限大小随机预言机, 例如从 $\mathcal{U}(\lambda \rightarrow \sigma)$ 中采样, 输入长度为 λ 。使用无限大小随机预言机的记号使我们免去了显式给出该上界的麻烦。

Remark 2.1.4. 均匀随机串模型 (URSM) 是这样一种设置: 所有参与方 (诚实方与恶意方) 都能访问一条多项式大小的公共随机串。该模型与 ROM 的区别在于随机串的大小: ROM 可被看作所有人都能访问同一条指数大小的随机串, 而 URSM 仅允许多项式大小的随机串。URSM 比 ROM 要弱, 但不允许构造具有无条件安全性的简洁论证。(参见??。)

✚

2.2 通过惰性采样进行模拟

随机预言机是一个无法高效整体采样的对象。尽管如此, 随机算法可以通过跨查询维护状态来高效地模拟随机预言机的输入-输出行为。我们称此为随机预言机的 **lazy sampling**。

从 $\mathcal{U}(\sigma)$ 采样的随机预言机会为每个输入查询独立分配一个随机的 $\{0, 1\}^\sigma$ 输出。模拟算法维护一个过去查询-回答对的列表。对于每个收到的查询, 如果该查询已在列表中出现, 则算法返回对应的回答; 否则, 算法从 $\{0, 1\}^\sigma$ 中采样一个随机回答, 将新的查询-回答对加入列表, 然后返回此回答。

从形式上看, 通过惰性采样从 $\mathcal{U}(\sigma)$ 生成预言机相当于下面这个带状态的随机算法实例:

- **初始化:** 将内部状态 S 设为空映射。
- **回答查询 x :** 如果 S 包含键 x , 则取 $y := S[x]$; 否则, 采样 $y \leftarrow \{0, 1\}^\sigma$, 并令 $S[x] := y$ 。输出 y 。

在全书中, 我们使用

$$\text{“lazily sample an oracle } \dot{f} \leftarrow \mathcal{U}(\sigma)\text{”}$$

这一说法, 并用符号 $\dot{f}$ 表示上述随机算法的一个新实例。记号 $\dot{f}$ 既能像其它预言机那样使用, 又提醒读者该预言机是惰性采样得到的。

注意, 惰性采样并非随机预言机的真实实现, 因为每个独立执行惰性采样的参与方对相同查询会获得不同的回答 (它们各自使用独立的随机性)。

然而, 惰性采样价值重大。首先, 它表明随机预言机具有简单的结构; 事实上, 并非所有函数分布都允许 (高效的) 惰性采样。²其次, 在安全归约中, 惰性采样是一个常用成分。在安全归约中,

²这里给出一个简单但退化的例子; 还有其它更有趣的例子。给定表 $g: \{0, 1\}^\sigma \rightarrow \{0, 1\}^\sigma$, 考虑分布 $\mathcal{D}_g$, 其以概率 1 分配到函数 $f: \{0, 1\}^* \rightarrow \{0, 1\}^\sigma$, 该 f 对 $\{0, 1\}^\sigma$ 中的查询按照表 g 回答, 其它查询回答 0。随机表 $g: \{0, 1\}^\sigma \rightarrow \{0, 1\}^\sigma$ 的最短描述长度通常为 $\Omega(2^\sigma)$, 此时 $\mathcal{D}_g$ 无法进行高效的惰性采样。由此可见, 存在无高效惰性采样的函数分布。

算法可能调用另一个期望访问随机预言机的子算法。如果归约算法可以将所有查询转发给一个外部预言机实例，就无需惰性采样；但有时外部预言机在归约设置中并不存在，或者归约算法需要对部分查询使用外部预言机，对另一些查询使用惰性采样，并将后者与其它量进行关联。全书中我们在多个安全归约中多次使用惰性采样。

2.3 通过惰性采样进行模拟

随机预言机是一个无法高效整体采样的对象。尽管如此，随机算法可以通过跨查询维护状态来高效地模拟随机预言机的输入-输出行为。我们称此为随机预言机的 *lazy sampling*。

从 $\mathcal{U}(\sigma)$ 采样的随机预言机会为每个输入查询独立分配一个随机的 $\{0, 1\}^\sigma$ 输出。模拟算法维护一个过去查询-回答对的列表。对于每个收到的查询，如果该查询已在列表中出现，则算法返回对应的回答；否则，算法从 $\{0, 1\}^\sigma$ 中采样一个随机回答，将新的查询-回答对加入列表，然后返回此回答。

从形式上看，通过惰性采样从 $\mathcal{U}(\sigma)$ 生成预言机相当于下面这个带状态的随机算法实例：

- 初始化：将内部状态 S 设为空映射。
- 回答查询 x ：如果 S 包含键 x ，则取 $y := S[x]$ ；否则，采样 $y \leftarrow \{0, 1\}^\sigma$ ，并令 $S[x] := y$ 。输出 y 。

在全书中，我们使用

“lazily sample an oracle $\dot{f} \leftarrow \mathcal{U}(\sigma)$ ”

这一说法，并用符号 $\dot{f}$ 表示上述随机算法的一个新实例。记号 $\dot{f}$ 既能像其它预言机那样使用，又提醒读者该预言机是惰性采样得到的。

注意，惰性采样并非随机预言机的真实实现，因为每个独立执行惰性采样的参与方对相同查询会获得不同的回答（它们各自使用独立的随机性）。

然而，惰性采样价值重大。首先，它表明随机预言机具有简单的结构；事实上，并非所有函数分布都允许（高效的）惰性采样。³其次，在安全归约中，惰性采样是一个常用成分。在安全归约中，算法可能调用另一个期望访问随机预言机的子算法。如果归约算法可以将所有查询转发给一个外部预言机实例，就无需惰性采样；但有时外部预言机在归约设置中并不存在，或者归约算法需要对部分查询使用外部预言机，对另一些查询使用惰性采样，并将后者与其它量进行关联。全书中我们在多个安全归约中多次使用惰性采样。

2.4 伪随机函数 vs. 随机预言机

伪随机函数是随机预言机的一种密码学实现，对现代密码学具有重要意义。下面我们回顾并讨论其定义。要点是：由于伪随机函数的安全性要求其（描述）保持秘密，因此在本书中并不重点使用伪随机函数。

伪随机函数的安全性质在于，其输入-输出行为在计算上与随机预言机不可区分。对手只能看到输入-输出行为，且并不接收函数的描述。为简单起见，我们在此使用 Theorem 2.1.2 中讨论的固定输入长度的随机预言机定义。

³这里给出一个简单但退化的例子；还有其它更有趣的例子。给定表 $g: \{0, 1\}^\sigma \rightarrow \{0, 1\}^\sigma$ ，考虑分布 $\mathcal{D}_g$ ，其以概率 1 分配到函数 $f: \{0, 1\}^* \rightarrow \{0, 1\}^\sigma$ ，该 f 对 $\{0, 1\}^\sigma$ 中的查询按照表 g 回答，其它查询回答 0 ^{σ} 。随机表 $g: \{0, 1\}^\sigma \rightarrow \{0, 1\}^\sigma$ 的最短描述长度通常为 $\Omega(2^\sigma)$ ，此时 $\mathcal{D}_g$ 无法进行高效的惰性采样。由此可见，存在无高效惰性采样的函数分布。

定义 2.4.1. 伪随机函数族具有误差 ϵ , 是指族 $\{F_{\lambda,\sigma}\}_{\lambda,\sigma \in \mathbb{N}}$, 其中每个 $F_{\lambda,\sigma}$ 是关于函数 $f: \{0,1\}^\lambda \rightarrow \{0,1\}^\sigma$ 的分布, 使得对于任意输入长度 λ 、输出长度 σ 和运行时间为 t 的预言机算法 A , 以下两种分布在统计距离上 $\epsilon(\lambda, \sigma, t)$ -接近:

$$\left\{ A^f \mid f \leftarrow \mathcal{U}(\lambda \rightarrow \sigma) \right\} \quad \text{and} \quad \left\{ A^f \mid f \leftarrow F_{\lambda,\sigma} \right\}.$$

该函数族还应能高效采样和高效求值: 存在多项式时间算法可在 $\text{poly}(\lambda, \sigma)$ 时间内采样 f , 且给定采样得到的 f 与输入 x , 可在 $\text{poly}(\lambda, \sigma)$ 时间内计算 $f(x)$ 。

伪随机函数是对随机预言机的高效去随机化。采样一个随机预言机需为 2^λ 个输入各自采样 σ 位随机串, 共需 $2^\lambda \cdot \sigma$ 位随机数; 而采样伪随机函数仅需多项式量的随机数。

伪随机函数适用于多个共享同一秘密的参与方希望同步访问同一份指数大小随机串的场景。他们可以用 $F_{\lambda,\sigma}$ 中采样得到的 f 对相关输入求值, 以对外展现为随机 (其他方仅知输入-输出对)。

例如, 伪随机函数可构造对称加密方案, 其实为一次性密钥本的去随机化: 通信双方预先共享 $f: \{0,1\}^\sigma \rightarrow \{0,1\}^\sigma$, 加密时采样随机盐 r 并发送 $c = (r, f(r) \oplus m)$, 可证明该方案满足选明文攻击安全性。

伪随机函数是“轻量”对象, 可由单向函数等基本假设或离散对数、学习误差等群和格假设构造。相比之下, 从理论角度看, ROM 是一个强理想模型, 在形式化安全性方面存在显著限制 (见 Section 2.5)。

2.4.1 实现方式

ROM 是一个理想化模型, 实际中需要高效实现。下面非正式讨论几种实现方式: 在 Section 2.4.2 中讨论最常用的基于密码学 Hash 函数的启发式实现; 在 Section 2.4.3 中讨论虽不常用但有时相关的其它实现。

2.4.2 基于密码学 Hash 函数

随机预言机通常启发式地由特定 Hash 函数实现, 后者被认为在输入-输出行为上足够近似随机预言机, 以保持某些协议的安全性。下面讨论该启发式及常见实例。

ROM 启发式 设协议 $\mathbb{P}$ 使用随机预言机 $f: \{0,1\}^* \rightarrow \{0,1\}^\sigma$ 。取高效可计算函数 $g: \{0,1\}^* \rightarrow \{0,1\}^\sigma$, 其计算时间为多项式。用 g 替换 f : (1) 公布 g 的计算算法; (2) 在协议中将所有对 f 的查询替换为对 g 的调用。得到新协议 $\mathbb{P}'$, 运行于公共参考字符串模型, 但一般不继承原 ROM 下的安全证明。通常我们假设 $\mathbb{P}'$ 安全, 称为协议 $\mathbb{P}$ 与 Hash 函数 g 的 ROM 启发式。

怎样的 Hash 函数合适? 应保证 g 近似满足随机预言机的若干安全属性, 如:

- **抗预像性:** 给定输出 y , 计算 x 使 $g(x) = y$ 不可行;
- **抗碰撞性:** 找到 $x_1 \neq x_2$ 使 $g(x_1) = g(x_2)$ 困难;
- **前缀抗性:** 找到 x 使 $g(x)$ 以许多零开头困难。

更一般地, 要确保 g 没有可利用结构, 对其仅能进行黑盒式通用攻击。

主流 Hash 函数 现实中有多种广泛用于实现 ROM 的 Hash 函数, 如 SHA3、BLAKE3。它们输出多种比特长度, 例如 SHA3-256、SHA3-512。在 x86 架构上, SHA3-256 的吞吐可低于每字节 15 个时钟周期。

这些函数具有:

- 设计原理成熟；
- 在 SSL/TLS、SSH、IPsec、Bitcoin 等高价值系统中应用，经久未破；
- 社区和标准化组织（如 NIST）大量研究和审视。

2.4.3 其它方法

虽 ROM 启发式最常用，还有以下假设性方案：(1) 可信中心；(2) 硬件 Token；(3) 软件混淆。这些方法或依赖伪随机函数，或在实践中效率、信任或可用性受限，不及 Hash 启发式。

2.5 优点与缺点

优点

- 数学简单：仅需理解随机函数，无需抽象代数。
- 具体安全：可给出微小常数的误差界，参数设置直观。
- 高效实现：基于 SHA3 等 Hash 函数，性能优良。
- 广泛适用：ROM 下构造的简洁论证无需其它假设；基于 ROM 的公钥加密、签名、FHE 等也已成库。
- 量子安全：可扩展到 QROM，已知的 Hash 函数对量子攻击也具备抗性。

缺点

- 有时不可实现：存在在 ROM 安全但用任何高效函数都不安全的协议。
- 适用受限：纯查询受限的 ROM 不足以实现密钥协商、公钥加密、OT 等。

2.6 多个随机预言机的设置

有时我们需要使用多个输出长度可能不同的随机预言机。我们用记号

$$(f_i)_{i \in [k]} \leftarrow \mathcal{U}((\ell_i)_{i \in [k]})$$

表示对每个 $i \in [k]$ 独立采样 $f_i \leftarrow \mathcal{U}(\ell_i)$ 。我们用

$$b \stackrel{\text{tr}}{\leftarrow} A^{(f_i)_{i \in [k]}}(a)$$

来表示算法 A 在输入 a 且对 $(f_i)_{i \in [k]}$ 有查询访问时输出 b ，且其查询-回答对列表为 tr 。 tr 中的每一对都带有对应的预言机索引。⁴

本节其余内容说明如何避免显式采样多重随机预言机。具体地，可通过域分离和输出扩展，从单个随机预言机 $f \leftarrow \mathcal{U}(\sigma)$ 推导出 $(f_i)_{i \in [k]}$ 。

域分离 例如，若希望得到两份独立的 $\mathcal{U}(\sigma)$ 随机预言机 f_0, f_1 ，可定义

$$f_0(x) := f((0, x)), \quad f_1(x) := f((1, x)).$$

前缀 0, 1 将两个域分离：任何对 f_0, f_1 的查询都映射到不同的 f 查询。一般地，取 i 为 $\lceil \log_2 k \rceil$ 位前缀，则对每个 $i \in [k]$ 定义

$$f_i(x) := f((i, x)),$$

即可域分离推导出 k 个独立随机预言机。

⁴形式上， tr 是三元组 (i, x, y) 的有序列表，表示对 f_i 的 x 查询得到 y 。

输出扩展 若需要输出长度 $\ell \neq \sigma$ 的预言机，可按以下方式处理：- 当 $\ell < \sigma$ 时，对查询答案取前 ℓ 位。- 当 $\ell > \sigma$ 时，令 $m = \lceil \ell / \sigma \rceil$ ，先通过域分离得出 $f_1, \dots, f_m$ ，再将各自答案拼接，最后视情况对最后一块进行截断。

具体而言，查询 x 的输出为

$$f_1(x) \parallel \dots \parallel \begin{cases} f_m(x), & \sigma \mid \ell, \\ \text{首 } (\ell \bmod \sigma) \text{ 位 of } f_m(x), & \text{否则.} \end{cases}$$

综合：域分离与扩展 更一般地，可对每个 $i \in [k]$ ，用记号

$$f_i := g^f(i, \ell_i)$$

定义推导规则：

- 令 $m_i = \lceil \ell_i / \sigma \rceil$;
- 对每 $j \in [m_i]$ ，设 $y_j = f((i, j, x))$;
- 若 $\ell_i \bmod \sigma \neq 0$ ，则对最后一块 y_{m_i} 取其前余数位；
- 输出 $y_1 \parallel \dots \parallel y_{m_i}$ 。

其中， i, j 分别为各自所需的位长前缀，用于域分离。

算法转换 可定义转换过程 M2S（多 $\rightarrow$ 单）和 S2M（单 $\rightarrow$ 多），将多预言机算法与单预言机算法相互转换，并严格分析查询次数与输出分布——详见本书第 ?? 章。

3 基本密码学性质

我们讨论随机预言机的基本密码学性质：

- 不可预测性 (Section 3.1)；
- 抗预像性 (Section 3.2)；
- 抗碰撞性 (Section 3.3)。

这些性质有助于熟悉随机预言机模型 (ROM) 中的基本安全性证明，并且在本书中各类构造的安全性分析中都会用到它们。

3.1 不可预测性

查询随机预言机 f 的算法在每次查询时都会获得一个查询-回答对。预测尚未进行的查询的回答 (成功概率超过随机猜测) 是不可能的，下面我们具体讨论。

考虑以下预测问题：给定对随机预言机 $f \leftarrow \mathcal{U}(\sigma)$ 的查询访问，输出一对 (x, y) ，使得 $f(x) = y$ ，且 (x, y) 并非通过查询 f 得到的已有查询-回答对。

由于随机预言机 $f \leftarrow \mathcal{U}(\sigma)$ 的定义，无论之前有多少查询，对 f 的任何新查询都会返回一个独立均匀的 $\{0, 1\}^\sigma$ 随机字符串。因此，无论攻击者拥有多少计算或查询资源，其在上述预测问题中的成功概率至多为 $\frac{1}{2^\sigma}$ 。下面给出该事实的形式化陈述与证明。

引理 3.1.1. 对于任意输出长度 $\sigma \in \mathbb{N}$ 、查询上界 $t \in \mathbb{N}$ 以及最多进行 t 次查询并输出候选查询-回答对 (x, y) 的算法 A ，都有

$$\Pr \left[\begin{array}{l} (x, y) \notin \text{tr} \\ \wedge f(x) = y \end{array} \middle| \begin{array}{l} f \leftarrow \mathcal{U}(\sigma), \\ (x, y) \stackrel{\text{tr}}{\leftarrow} A^f \end{array} \right] \leq \frac{1}{2^\sigma}.$$

Proof. 令 E 表示事件“ A 对 x 进行了查询”。我们将其分为两种不相交的情况来讨论，并由此得到所需的上界。

- 情况 1: A 查询了 x 。

若 A 在 x 上进行了查询，且 $(x, y) \notin \text{tr}$ ，则必有 $f(x) \neq y$ 。因此

$$\Pr \left[(x, y) \notin \text{tr} \wedge f(x) = y \wedge E \middle| f \leftarrow \mathcal{U}(\sigma), (x, y) \stackrel{\text{tr}}{\leftarrow} A^f \right] = 0.$$

- 情况 2: A 未查询 x 。

若 A 未对 x 进行查询 (此时 $(x, y) \notin \text{tr}$)，则 $f(x)$ 与 tr 独立，而 y 仅依赖于 tr (及 A 的私有随机性)，故 $f(x)$ 与 y 相互独立。于是

$$\Pr \left[(x, y) \notin \text{tr} \wedge f(x) = y \wedge \neg E \middle| f \leftarrow \mathcal{U}(\sigma), (x, y) \stackrel{\text{tr}}{\leftarrow} A^f \right] = \Pr[y' = y \mid y' \leftarrow \{0, 1\}^\sigma] = \frac{1}{2^\sigma}.$$

综上，两种情况的概率之和不超过 $\frac{1}{2^\sigma}$ ，证明完成。 $\square$

3.2 抗预像性

考虑以下反演问题：给定对随机预言机 $f \leftarrow \mathcal{U}(\sigma)$ 的查询访问，以及给定目标 $y \in \{0, 1\}^\sigma$ ，找出一个输入 $x \in \{0, 1\}^*$ ，使得 $f(x) = y$ 。

由于对随机预言机的任何一次查询都会返回一个均匀随机的 $\{0, 1\}^\sigma$ 字符串，因此解决该问题的自然“攻击”策略是：在查询预算为 t 的情况下，随意选择 $x_1, \dots, x_t$ 互不相同的查询，并查看是否有查询的回答等于 y ——若有则输出该查询，否则放弃。每次查询回答等于 y 的概率恰为 $\frac{1}{2^\sigma}$ ，共进行 t 次查询。由容斥原理（见 Theorem 1.1.2），反演目标 y 的成功概率可下界为：

$$\begin{aligned} & \Pr[\exists i \in [t] : f(x_i) = y] \\ & \geq \sum_{i \in [t]} \Pr[f(x_i) = y] - \sum_{\substack{i, j \in [t] \\ i \neq j}} \Pr[f(x_i) = y \wedge f(x_j) = y] \\ & = \frac{t}{2^\sigma} - \binom{t}{2} \cdot \frac{1}{2^{2\sigma}} \geq \frac{1}{2} \cdot \frac{t}{2^\sigma}. \end{aligned}$$

上述成功概率与 t 呈线性增长，这印证了更多查询会提升反演概率的直观认识。然而，当 t 为 σ 的某个多项式函数时，该成功概率仍是 σ 的一个可忽略函数。

下列引理表明上述策略在本质上是最优的：任何 t -查询算法要使其某次查询映射到目标 y 的概率，均上界为 $O(\frac{t}{2^\sigma})$ 。这说明随机预言机具有抗预像性（在密码学中有时称为“一向性”）。

引理 3.2.1. 对于任意输出长度 $\sigma \in \mathbb{N}$ 、查询上界 $t \in \mathbb{N}$ 、最多进行 t 次查询的算法 A 及目标 $y \in \{0, 1\}^\sigma$ ，都有

$$\Pr[\exists x \in \text{tr} : f(x) = y \mid f \leftarrow \mathcal{U}(\sigma), \perp \stackrel{\text{tr}}{\leftarrow} A^f] \leq \frac{t}{2^\sigma}.$$

Proof. 不失一般性，假设 A 的查询两两不同。对每次第 i 次查询，定义指示变量 X_i ，当第 i 次查询的回答等于 y 时 $X_i = 1$ 。因为任何未曾查询的输入均返回独立均匀的 $\{0, 1\}^\sigma$ 随机串，所以

$$\Pr[X_i = 1] = \frac{1}{2^\sigma} \quad \forall i \in [t].$$

由并集上界（union bound）可得

$$\Pr[\exists i : X_i = 1] \leq \sum_{i=1}^t \Pr[X_i = 1] = t \cdot \frac{1}{2^\sigma} = \frac{t}{2^\sigma}.$$

这正是所需的不等式。 $\square$

上述引理的结论在以下扩展情形下同样成立：若目标 y 按任意分布采样，或由对手在无查询访问的预计算阶段选定，则反演成功概率依旧受相同上界约束——这由概率分布的凸性立刻得到。

若目标 y 基于随机预言机选取，对手可 trivially 选择某固定查询 x ，令 $y = f(x)$ ，从而必然反演成功。我们可考虑另一情形：目标 y 源自一个对手未知、但与预言机无关的随机查询 x 。下式给出对这种高熵目标的上界。

引理 3.2.2. 对于任意输出长度 $\sigma \in \mathbb{N}$ 、查询上界 $t \in \mathbb{N}$ 、最多进行 t 次查询的算法 A 以及参数 $n \in \mathbb{N}$ ，都有

$$\begin{aligned} & f \leftarrow \mathcal{U}(\sigma), \\ & \Pr\left[(x, y) \in \text{tr} \mid x \leftarrow \{0, 1\}^n, y \leftarrow f(x), \right. \\ & \quad \left. \perp \stackrel{\text{tr}}{\leftarrow} A^f(y) \right] \leq \frac{t}{2^n}. \end{aligned}$$

Proof. 因为 $y = f(x)$ 在 $\{0, 1\}^\sigma$ 中均匀分布, 我们可将实验等价替换为: 先执行对手的查询阶段及输出阶段, 再由实验在 $\{0, 1\}^\sigma$ 中采样 y , 最后在 $\{0, 1\}^n$ 中采样 x 。此时 $(x, y) \in \text{tr}$ 当且仅当随机采样的 x 刚好与对手查询轨迹中某个查询相同, 而这概率至多为 $\frac{2^n}{2^n} \cdot \frac{1}{2^n} = \frac{t}{2^n}$ 。故证毕。 $\square$

我们进一步推广到拥有多目标的情形: 对手需破译目标列表 $y_1, \dots, y_k$, 其中每个目标对应不同的随机预言机 f_i 。

引理 3.2.3. 对于任意输出长度 $\sigma \in \mathbb{N}$ 、查询上界 $t \in \mathbb{N}$ 、最多进行 t 次查询的算法 A 、参数 $n \in \mathbb{N}$ 以及目标数量 $k \in \mathbb{N}$, 都有

$$\Pr \left[\forall i \in [k], (x_i, y_i) \in \text{tr} \mid \begin{array}{l} \forall i: f_i \leftarrow \mathcal{U}(\sigma), \\ x_i \leftarrow \{0, 1\}^n, y_i \leftarrow f_i(x_i), \\ \perp \stackrel{\text{tr}}{\leftarrow} A^{f_1, \dots, f_k}(y_1, \dots, y_k) \end{array} \right] \leq \left(\frac{t}{2^n} \right)^k.$$

Proof. 与上一引理证明思路相同: 对每个 i , y_i 于 $\{0, 1\}^\sigma$ 中均匀分布, 可延后采样, 再由随机采样的 x_i 与查询轨迹比对。各 i 之间相互独立, 故联合反演所有目标的概率上界即为 $\left(\frac{t}{2^n}\right)^k = \left(\frac{t}{2^n}\right)^k$ 。 $\square$

3.3 抗碰撞性

考虑以下碰撞问题: 给定对随机预言机 $f \leftarrow \mathcal{U}(\sigma)$ 的查询访问, 寻找两个互不相同的输入 $x, x' \in \{0, 1\}^*$, 使得

$$f(x) = f(x').$$

与反演问题类似, 解决碰撞问题的自然“攻击”策略是: 在查询预算为 t 的情况下, 向预言机提出 t 个任意且两两不同的查询, 并将其中产生的任何碰撞作为输出——若未找到则放弃。不同于反演问题, 该策略的成功概率呈二次增长, 理由如下:

对任意两次不同查询 $x_i \neq x_j$, 有

$$\Pr[f(x_i) = f(x_j)] = \frac{1}{2^\sigma}.$$

总共有 $\binom{t}{2}$ 对查询, 因此此策略的成功概率为

$$\Omega\left(\frac{t^2}{2^\sigma}\right),$$

这就是所谓的“生日攻击”现象。尽管如此, 当 t 为 σ 的多项式时, 该成功概率仍是可忽略的。

下面的引理表明, 任何 t -查询算法找到碰撞的概率都至多为 $O(t^2/2^\sigma)$, 即随机预言机具有抗碰撞性。

引理 3.3.1. 对于任意输出长度 $\sigma \in \mathbb{N}$ 、查询上界 $t \in \mathbb{N}$ 及最多进行 t 次查询的算法 A , 都有

$$\Pr \left[\exists (x, y), (x', y) \in \text{tr} \text{ 且 } x \neq x' \mid f \leftarrow \mathcal{U}(\sigma), \perp \stackrel{\text{tr}}{\leftarrow} A^f \right] \leq \frac{1}{2} \cdot \frac{(t-1) \cdot t}{2^\sigma}.$$

Proof. 不妨令 A 的查询序列为 $x_1, \dots, x_t$ 。对每对 (i, j) , $i \neq j$, 定义事件

$$E_{i,j}: x_i \neq x_j \quad \wedge \quad f(x_i) = f(x_j).$$

显然 $\Pr[E_{i,j}] \leq \frac{1}{2^\sigma}$ 。由并集上界 (union bound) 得

$$\Pr[\exists i \neq j: E_{i,j}] \leq \sum_{i \neq j} \Pr[E_{i,j}] = \binom{t}{2} \cdot \frac{1}{2^\sigma} = \frac{1}{2} \cdot \frac{(t-1) \cdot t}{2^\sigma}.$$

这即是所需的不等式。 $\square$

下述命题表明，上述上界在常数因子内是紧的。

断论 3.3.2. 令输出长度 $\sigma \in \mathbb{N}$ 、查询上界 $t \in \mathbb{N}$ ，且 $t > 1$ 且 $\frac{t^2}{2^\sigma} \leq \frac{1}{4}$ 。则存在一 t -查询算法 A ，其成功找到碰撞的概率满足

$$\Pr\left[\exists (x, y), (x', y) \in \text{tr} \text{ 且 } x \neq x' \mid f \leftarrow \mathcal{U}(\sigma), \perp \stackrel{\text{tr}}{\leftarrow} A^f\right] \geq \frac{1}{10} \frac{t^2}{2^\sigma}.$$

Proof. 令 A 对随机预言机进行任意 t 个两两不同的查询。与上文类似，对每对 (i, j) 定义事件 $E_{i,j}$ 。由于查询两两不同， $\Pr[E_{i,j}] = 1/2^\sigma$ 。应用包含-排除原理（三阶与四阶项的具体系数使用二项式系数近似，可参见 Theorems 1.1.2 and 1.3.1），在假设 $t^2/2^\sigma \leq 1/4$ 下，可得

$$\Pr\left[\bigvee_{i \neq j} E_{i,j}\right] \geq \frac{t^2}{10 \cdot 2^\sigma}.$$

详见上述证明。 □

3.4 交互式论证

非交互式论证的定义可以放宽，允许论证者与验证者在若干轮中交换消息。我们引入此概念，因为本书有时会先研究更简单的交互式协议，再讨论其非交互式对应物。

在 ROM 中，一个交互式论证 (IARG) 由以下元组给出：

$$\text{IARG} = (\mathcal{P}, \mathcal{V})$$

其中 $\mathcal{P}$ 是一个带预言机访问的交互式算法，称为论证者， $\mathcal{V}$ 是另一个带预言机访问的交互式算法，称为验证者。

设随机预言机 $f \leftarrow \mathcal{U}(\sigma)$ ，输出长度为 $\sigma \in \mathbb{N}$ 。包括论证者和验证者在内的任何人都可以查询 f 。论证者 $\mathcal{P}$ 接收实例 $\mathbb{x}$ 和见证 $\mathbb{w}$ ，验证者 $\mathcal{V}$ 接收实例 $\mathbb{x}$ 。双方在若干轮内交互，交互结束后，验证者输出一比特，1 表示接受，0 表示拒绝（见图 ??）。

我们说 $\text{IARG} = (\mathcal{P}, \mathcal{V})$ 是关系 $\mathcal{R}$ 的 ROM 交互式论证，当且仅当满足以下两大性质：完备性和可靠性。

- **完备性。** 对于每个 $(\mathbb{x}, \mathbb{w}) \in \mathcal{R}$ ，当论证者和验证者皆诚实时，验证者必然接受。即对于任意 σ ,

$$\Pr\left[\langle \mathcal{P}^f(\mathbb{x}, \mathbb{w}), \mathcal{V}^f(\mathbb{x}) \rangle_{\text{ARG}} = 1 \mid f \leftarrow \mathcal{U}(\sigma)\right] = 1.$$

概率越过 f 以及双方的私有随机性（见定义 3.4.1）。

- **可靠性。** 对于任意 σ 、查询上界 t 、至多 t 次查询的恶意论证者 $\tilde{\mathcal{P}}$ 以及任何 $\mathbb{x} \notin \mathcal{L}(\mathcal{R})$ ，验证者接受的概率至多为小量 ϵ_{ARG} 。

- **非自适应可靠性：** 实例 $\mathbb{x}$ 在预言机采样前固定（见定义 3.4.2）。
- **自适应可靠性：** 恶意论证者可在查询预言机后自适应选择实例（见定义 3.4.3）。

以下给出形式化定义；记 $\langle A, B \rangle_{\text{ARG}}$ 表示 B 与 A 交互后的输出结果随机变量。

 figure:iarg-diagram

Definition 3.4.1: 完备性

交互式论证 $\text{IARG} = (\mathcal{P}, \mathcal{V})$ 对于关系 $\mathcal{R}$ 若满足: 对任意 $\sigma \in \mathbb{N}$ 及 $(\mathbb{x}, \mathbb{w}) \in \mathcal{R}$,

$$\Pr[\langle \mathcal{P}^f(\mathbb{x}, \mathbb{w}), \mathcal{V}^f(\mathbb{x}) \rangle_{\text{ARG}} = 1 \mid f \leftarrow \mathcal{U}(\sigma)] = 1.$$

Definition 3.4.2: 非自适应可靠性

交互式论证 $\text{IARG} = (\mathcal{P}, \mathcal{V})$ 若满足: 对任意 $\sigma \in \mathbb{N}$ 、 $t \in \mathbb{N}$ 、 t -查询的 $\tilde{\mathcal{P}}$ 及 $\mathbb{x} \notin \mathcal{L}(\mathcal{R})$,

$$\Pr[\langle \tilde{\mathcal{P}}^f, \mathcal{V}^f(\mathbb{x}) \rangle_{\text{ARG}} = 1 \mid f \leftarrow \mathcal{U}(\sigma)] \leq \epsilon_{\text{ARG}}(\sigma, t, \mathbb{x}).$$

Definition 3.4.3: 自适应可靠性

交互式论证 $\text{IARG} = (\mathcal{P}, \mathcal{V})$ 若满足: 对任意 $\sigma \in \mathbb{N}$ 、 $t \in \mathbb{N}$ 、 t -查询的 $\tilde{\mathcal{P}}$ 及实例长度上界 $n \in \mathbb{N}$,

$$\Pr \left[\begin{array}{l} |\mathbb{x}| \leq n, \mathbb{x} \notin \mathcal{L}(\mathcal{R}), \\ \langle \tilde{\mathcal{P}}^f(\text{aux}), \mathcal{V}^f(\mathbb{x}) \rangle_{\text{ARG}} = 1 \end{array} \mid \begin{array}{l} f \leftarrow \mathcal{U}(\sigma), \\ (\mathbb{x}, \text{aux}) \leftarrow \tilde{\mathcal{P}}^f \end{array} \right] \leq \epsilon_{\text{ARG}}(\sigma, t, n).$$

公开掷币 若验证者所有消息均为独立均匀随机比特串, 则称该论证公开掷币 (public-coin) (见定义 ??)。非交互式论证天然满足此条件。

4 附加的安全性定义

在实际应用中，论证通常需要满足除了完备性和可靠性之外的额外安全性定义。由于本书重点关注非交互式论证（NARG），下文仅针对 NARG 给出这些附加定义。

- 在 Section 4.1 中讨论知识可靠性。
- 在 Section 4.2 中讨论零知识。

本书将说明如何为我们研究的每种 NARG 构造满足这些附加安全性定义。

4.1 知识可靠性

可靠性（见 ???）保证任何恶意论证者都无法以超过极小错误概率说服验证者接受不在语言中的实例。

知识可靠性则更强：它要求当论证者成功说服验证者接受时（除去极小错误概率），该论证者“确实知道”一个合法见证证明实例属于语言。¹

若实例属于语言，可通过暴力搜索找到见证，但效率低下。知识可靠性要求存在一个高效的提取器，能从论证者的行为中提取出见证，从而体现论证者的“知识”。

针对提取器访问恶意论证者的方式，有不同定义。下文给出两种自适应形式的知识可靠性定义——提取器均可在论证者选择实例后才作工作。²

直线提取（Straightline knowledge soundness） 此强定义要求提取器仅利用一次执行的输出与查询-回答轨迹完成提取，因此在不可编程 ROM 中成立。

Definition 4.1.1: 直线知识可靠性

令非交互式论证 $\text{NARG} = (\mathcal{P}, \mathcal{V})$ 针对关系 $\mathcal{R}$ 具有**直线知识可靠性误差** κ_{ARG} ，当存在多项式时间确定性算法 $\mathcal{E}$ （提取器），使得对任意

$$\sigma \in \mathbb{N}, \quad t \in \mathbb{N}, \quad \tilde{\mathcal{P}}(t\text{-查询}, \text{确定性}), \quad n \in \mathbb{N},$$

均有

$$\Pr \left[|\mathbb{X}| \leq n, (\mathbb{X}, \mathbb{W}) \notin \mathcal{R}, \mathcal{V}^f(\mathbb{X}, \pi) = 1 \mid \begin{array}{l} f \leftarrow \mathcal{U}(\sigma), \\ (\mathbb{X}, \pi) \xleftarrow{\text{tr}} \tilde{\mathcal{P}}^f, \\ \mathbb{W} \leftarrow \mathcal{E}(\mathbb{X}, \pi, \text{tr}) \end{array} \right] \leq \kappa_{\text{ARG}}(\sigma, t, n).$$

¹满足此属性的论证有时称为知识证明，若强调仅对受限的恶意论证者有效则称为知识论证。

²对应的非自适应版本可从下述定义中直接推导。

重放提取 (Rewinding knowledge soundness) 此定义允许提取器在提取时重放 (rewind) 或多次运行恶意论证者, 并访问随机预言机, 以增强提取能力。错误概率还可依赖于论证者使验证者接受的成功概率。

定义 4.1.1. 令非交互式论证 $\text{NARG} = (\mathcal{P}, \mathcal{V})$ 。一个确定性论证者 $\tilde{\mathcal{P}}$ 的失败概率 $\delta_{\tilde{\mathcal{P}}}$ 定义为: 对任意

$$\sigma \in \mathbb{N}, \quad n \in \mathbb{N},$$

有

$$\Pr \left[|\mathbb{x}| > n \vee \mathcal{V}^f(\mathbb{x}, \pi) = 0 \mid \begin{array}{l} f \leftarrow \mathcal{U}(\sigma), \\ (\mathbb{x}, \pi) \leftarrow \tilde{\mathcal{P}}^f \end{array} \right] \leq \delta_{\tilde{\mathcal{P}}}(\sigma, n).$$

Definition 4.1.2: 重放知识可靠性

令非交互式论证 $\text{NARG} = (\mathcal{P}, \mathcal{V})$ 针对关系 $\mathcal{R}$ 具有重放知识可靠性误差 κ_{ARG} 及提取时长 et_{ARG} , 当存在概率算法 $\mathcal{E}$ (提取器), 使得对任意

$$\sigma \in \mathbb{N}, \quad t \in \mathbb{N}, \quad \tilde{\mathcal{P}}(t\text{-查询}, \text{确定性}, \text{失败概率 } \delta_{\tilde{\mathcal{P}}}, \text{运行时 } \tau_{\tilde{\mathcal{P}}}), \quad n \in \mathbb{N},$$

均有

$$\Pr \left[|\mathbb{x}| \leq n, (\mathbb{x}, \mathbb{w}) \notin \mathcal{R}, \mathcal{V}^f(\mathbb{x}, \pi) = 1 \mid \begin{array}{l} f \leftarrow \mathcal{U}(\sigma), \\ (\mathbb{x}, \pi) \leftarrow \tilde{\mathcal{P}}^f, \\ \mathbb{w} \leftarrow \mathcal{E}^f(\mathbb{x}, \pi, \text{tr}, \mathcal{O}_{\tilde{\mathcal{P}}}) \end{array} \right] \leq \kappa_{\text{ARG}}(\sigma, t, n, \delta_{\tilde{\mathcal{P}}}(\sigma, n)),$$

且 $\mathcal{E}$ 的期望运行时为

$$\text{et}_{\text{ARG}}(\sigma, t, n, \delta_{\tilde{\mathcal{P}}}(\sigma, n), \tau_{\tilde{\mathcal{P}}}(\sigma, n)).$$

Remark 4.1.2 (等价定义). 在本书中, 我们常用另一等价定义——提取器直接接收验证者的查询-回答轨迹, 而非访问随机预言机。两种定义等价, 理由如下: 提取器对论证者的查询轨迹已有记录, 而对未查询的输入, 提取器可自行随机回答; 仅需额外提供验证者的查询-回答轨迹。形式证明可参见正文。

4.2 零知识

可靠性和知识可靠性保护验证者免受恶意论证者攻击; 相对地, 零知识保护 (诚实) 论证者的隐私。

实例 $\mathbb{x}$ 是论证者和验证者的公共输入, 而见证 $\mathbb{w}$ 仅是论证者的私有输入。验证者并不直接获得 $\mathbb{w}$, 而是获得论证者输出的论证字符串 π , 这通常能显著提高效率: π 通常远小于 $\mathbb{w}$, 或验证 π 的时间远快于直接检验 $(\mathbb{x}, \mathbb{w}) \in \mathcal{R}$ 。

零知识要求 π 不泄露除 “ $\mathbb{x}$ 属于语言” 之外的任何 $\mathbb{w}$ 信息。形式上, 存在高效的概率算法 模拟器 S , 使任意对手 $\mathcal{A}$ 在两种实验中的输出统计不可区分:

- **真实世界:** $\mathcal{A}$ 接收由真实论证者生成的 π ;
- **模拟世界:** $\mathcal{A}$ 接收由模拟器仅凭实例 $\mathbb{x}$ 生成的 π , 同时模拟器在 EPROM 模型中预先 “编程” 随机预言机。

具体定义如下。

(1) **非自适应零知识** 这里假设实例 $(\mathbb{x}, \mathbb{w}) \in \mathcal{R}$ 在预言机采样前固定。对任意

$$\sigma \in \mathbb{N}, \quad t \in \mathbb{N}, \quad \mathcal{A}(t\text{-查询}), \quad (\mathbb{x}, \mathbb{w}) \in \mathcal{R},$$

定义两分布：

$$\mathcal{D}_{\text{real}} = \{ \text{out} \mid f \leftarrow \mathcal{U}(\sigma); \pi \leftarrow \mathcal{P}^f(\mathbb{x}, \mathbb{w}); \text{out} \leftarrow \mathcal{A}^f(\pi) \},$$

$$\mathcal{D}_{\text{sim}} = \{ \text{out} \mid f \leftarrow \mathcal{U}(\sigma); (\pi, \mu) \leftarrow \mathcal{S}^f(\mathbb{x}); \text{out} \leftarrow \mathcal{A}^{f[\mu]}(\pi) \}.$$

若这两分布在统计距离上不超过 $z_{\text{ARG}}(\sigma, t, \mathbb{x})$ ，则称 NARG 具有**非自适应零知识误差** z_{ARG} （参见定义 ??）。



definition:once-nonadaptive

(2) **自适应零知识** 对手先与预言机交互，选定实例、见证和内部状态后，再继续接收 π 。对任意

$$\sigma \in \mathbb{N}, \quad t \in \mathbb{N}, \quad \mathcal{A}(t\text{-查询}, \text{选择合法}), \quad n \in \mathbb{N},$$

定义两分布：

$$\mathcal{D}_{\text{real}} = \{ \text{out} \mid f \leftarrow \mathcal{U}(\sigma); (\mathbb{x}, \mathbb{w}, \text{aux}) \leftarrow \mathcal{A}^f; \pi \leftarrow \mathcal{P}^f(\mathbb{x}, \mathbb{w}); \text{out} \leftarrow \mathcal{A}^f(\text{aux}, \pi) \},$$

$$\mathcal{D}_{\text{sim}} = \{ \text{out} \mid f \leftarrow \mathcal{U}(\sigma); (\mathbb{x}, \mathbb{w}, \text{aux}) \leftarrow \mathcal{A}^f; (\pi, \mu) \leftarrow \mathcal{S}^f(\mathbb{x}); \text{out} \leftarrow \mathcal{A}^{f[\mu]}(\text{aux}, \pi) \}.$$

若两分布统计距离至多 $z_{\text{ARG}}(\sigma, t, n)$ ，则称具有**自适应零知识误差** z_{ARG} （参见定义 ??）。



definition:adaptive-zero-knowledge

见证不可区分性 零知识蕴含较弱的见证不可区分性：对同一实例，用不同见证生成的 π 分布几乎相同。形式定义如下。

对任意

$$\sigma \in \mathbb{N}, \quad t \in \mathbb{N}, \quad \mathcal{A}(t\text{-查询}, \text{输出合法}), \quad n \in \mathbb{N},$$

令

$$X_b = \{ \text{out} \mid f \leftarrow \mathcal{U}(\sigma); (\mathbb{x}, \mathbb{w}_0, \mathbb{w}_1, \text{aux}) \leftarrow \mathcal{A}^f; \pi \leftarrow \mathcal{P}^f(\mathbb{x}, \mathbb{w}_b); \text{out} \leftarrow \mathcal{A}(\text{aux}, \pi) \}.$$

若 X_0 与 X_1 在统计距离上不超 $z_{\text{ARG}}(\sigma, t, n)$ ，则称具有**自适应见证不可区分性误差** z_{ARG} （参见定义 ??）。



definition:adaptive-witness-indistinguishability

引理 4.2.1. 若某 NARG 具有**自适应零知识误差** z_{ARG} ，则其**见证不可区分性误差**至多 $2z_{\text{ARG}}$ 。

Bibliography

- [CG21] Alessandro Chiesa and Tom Gur
Foundations and frontiers of probabilistic proofs
2021.
URL: <https://www.slmath.org/workshops/931>.
(Cited on page ix.)
- [Chi23] Alessandro Chiesa
Foundations and frontiers of probabilistic proofs
2023.
URL: <https://www.slmath.org/workshops/1037>.
(Cited on page ix.)
- [Tha22] Justin Thaler
Proofs, arguments, and zero-knowledge
2022.
URL: <https://people.cs.georgetown.edu/jthaler/ProofsArgsAndZK.html>.
(Cited on page ix.)
- [ZKP22] ZKProof
ZKProof community reference
2022.
URL: <https://docs.zkproof.org/reference>.
(Cited on page ix.)